

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

A Importância das Ferramentas Open Source no Desenvolvimento de Pipelines de Dados

Pedro Medeiros Pereira Carvalho

Prof. Orientador:

Diego Cardoso Borda Castro

**Rio de Janeiro,
Dezembro de 2025**

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

A Importância das Ferramentas Open Source no Desenvolvimento de Pipelines de Dados

Pedro Medeiros Pereira Carvalho

Projeto final apresentado em cumprimento às
normas do Departamento de Educação
Superior do Centro Federal de Educação
Tecnológica Celso Suckow da Fonseca,
CEFET/RJ, como parte dos requisitos para
obtenção do título de Bacharel em Sistemas de
Informação.

Prof. Orientador:
Diego Cardoso Borda Castro

**Rio de Janeiro,
Dezembro de 2025**

AGRADECIMENTOS

Upcoming...

RESUMO

A criação de pipelines de dados por meio de soluções proprietárias apresenta algumas desvantagens competitivas, como a falta de transparência, a dependência de fornecedores e, principalmente, os altos custos. O *software* de código aberto surge nesse contexto com um modelo de desenvolvimento descentralizado e colaborativo, que distribui o código-fonte publicamente, permitindo que qualquer pessoa possa usar, alterar e redistribuir como preferir, sem custo adicional. No entanto, o mercado ainda enfrenta desafios e resistências na adoção dessas soluções. Para enfrentar esse problema, este trabalho mapeou os principais *softwares* livres utilizados no mercado para atividades de engenharia de dados, com o propósito de criar um catálogo de aplicações para construção de *pipeline* de dados *open source*. Com o intuito de demonstrar a viabilidade prática, foi desenvolvido um exemplo prático de *pipeline* com algumas das ferramentas encontradas, desde a concepção inicial até a implementação final, aplicado a um cenário de coleta de informações sobre passagens aéreas para o Brasil, extraindo as informações de diferentes sites internacionais. A análise focou no impacto dessas ferramentas em termos de escalabilidade e desempenho, com o intuito de oferecer uma base técnica relevante para trabalhos acadêmicos e pequenas e médias empresas que buscam soluções eficientes e acessíveis, sem comprometer seus recursos.

Palavras-chaves: *Pipeline* de Dados; *Software* Livre; Engenharia de Dados

ABSTRACT

The development of data pipelines utilising proprietary solutions entails several competitive drawbacks, including opacity, vendor dependency, and, crucially, elevated expenses. Open-source software arises inside this framework, characterised by a decentralised and collaborative development process that publicly disseminates the source code, permitting anybody to utilise, alter, and redistribute it at no additional cost. Nonetheless, the market continues to encounter obstacles and opposition in embracing these solutions. This study catalogued the primary free software utilised in the market for data engineering tasks, aiming to establish a repository of apps for constructing open-source data pipelines. To illustrate practical viability, a pipeline was constructed utilising various tools, from initial conception to final implementation, focused on gathering data regarding airline tickets to Brazil by extracting information from multiple international websites. The investigation examined the influence of these tools regarding scalability and performance, intending to furnish a pertinent technical foundation for academic research and small to medium-sized firms in pursuit of efficient and cost-effective solutions without depleting their resources.

Keywords: *Data Pipeline; Open Source; Data Engineering*

Acrónimos

API Application Programming Interface

BI Business Intelligence

CLI Command-Line Interface

CSRF Cross-site Request Forgery

DL Data Lake

DSR Design Science Research

DW Data Warehouse

ETL Extract, Transform, Load

HTML HyperText Markup Language

HTTP Hypertext Transfer Protocol

IoT Internet of Things

LLM Large Language Model

NPM Node Package Manager

ORM Object-Relational Mapping

PHP Hypertext Preprocessor

RR Rapid Review

SQL Structured Query Language

URL Uniform Resource Locator

Sumário

1	Introdução	2
1.1	Contextualização	2
1.2	Objetivos do Trabalho	4
1.3	Justificativas da Abordagem <i>Open Source</i> no <i>Pipeline</i> de Dados	4
1.3.1	Objetivos Específicos	5
1.4	Metodologia de Pesquisa: Design Science Research (DSR)	7
1.4.1	Fluxo Metodológico (DSR)	7
1.5	Organização do texto	8
2	Fundamentação Teórica	10
2.1	ETL e a Evolução das Arquiteturas de Dados	10
2.2	Open Source: Estratégia, Flexibilidade e Comparação	11
2.3	Técnicas Avançadas de Ingestão e Processamento	12
2.4	Tendências Emergentes: Reverse ETL	12
3	Revisão da Literatura	14
3.0.1	Artigos Selecionados para a Revisão	16
3.1	Respostas às Questões de Pesquisa	17
3.1.1	Quais são as principais ferramentas open source utilizadas no desenvolvimento de pipelines de dados atualmente?	18
3.1.2	De que forma o uso de ferramentas open source impacta a eficiência, flexibilidade e custo dos pipelines de dados?	19
3.1.3	Em quais contextos as ferramentas open source são mais vantajosas para o desenvolvimento de pipelines de dados?	20
3.1.4	Quais os benefícios e limitações do uso de ferramentas open source em comparação com ferramentas proprietárias no contexto da engenharia de dados?	21
4	Construção do Pipeline	23
4.0.1	Tentativa com a ferramenta Browser Use (<i>Web Scraping</i> com Inteligência Artificial)	26

4.1	Web Scraping	27
4.1.1	Diferenciação e Propósito	27
4.2	Ingestão dos dados	28
4.2.1	Limitações do Pandas e do Apache Spark	29
4.2.2	Modelagem das tabelas	30
4.2.3	Implementação Prática	31
4.2.4	Desafios e Estratégias de Mitigação	32
4.2.5	Formato e Armazenamento dos Dados Brutos	33
4.3	Transformação dos Dados com dbt e PostgreSQL	33
4.3.1	Migração do DuckDB para PostgreSQL	33
4.3.2	Arquitetura Medallion no PostgreSQL	34
4.3.3	Camada Staging: Limpeza e Padronização	35
4.3.4	Macros Reutilizáveis	37
4.3.5	Camada Marts: Consolidação e Modelagem Dimensional	38
4.3.6	Testes de Qualidade de Dados	40
4.3.7	Arquitetura da DAG	41
4.3.8	Uso do DockerOperator	42
4.3.9	Configuração do Agendamento	43
4.3.10	Configuração do Executor	43
4.3.11	Resultados da Orquestração	43
4.4	Visualização dos Dados com Apache Superset	46
4.4.1	Escolha e Configuração da ferramenta	46
4.4.2	Dashboards Desenvolvidos	46
4.4.3	Resultados da Visualização e Análise dos Dados	47
4.5	Infraestrutura Containerizada com Docker	51
5	Conclusão	54
5.1	Estimativa de Custos Operacionais	54
5.1.1	Premissas da Estimativa	54
5.1.2	Infraestrutura de Servidor	55
5.1.3	Armazenamento	56
5.1.4	Transferência de Dados	57
5.1.5	Custo Total e Comparação com Soluções Proprietárias	57

5.2	Análise Crítica e Lições Aprendidas	59
5.2.1	Limitações de Escalabilidade das Ferramentas <i>Open Source</i>	59
5.2.2	Síntese das Lições Aprendidas	60
	Referências Bibliográficas	61

Lista de Figuras

FIGURA 1:	Fluxo Metodológico do Projeto Mapeado às Fases do <i>Design Science Research</i> (DSR)	7
FIGURA 2:	Arquitetura completa do pipeline de dados	24
FIGURA 3:	Estrutura visual da DAG pipeline_destinos_brasil na interface do Apache Airflow, exibindo o encadeamento sequencial das 16 tarefas.	44
FIGURA 4:	Detalhes da execução da DAG, com status de sucesso e duração total de 19 minutos e 20 segundos.	44
FIGURA 5:	Painel <i>Panorama Geral</i> no Apache Superset	47
FIGURA 6:	Painel <i>Detalhamento por Destino</i> no Apache Superset	48

Capítulo 1

Introdução

Este capítulo tem como propósito introduzir o cenário em que se insere este trabalho de conclusão de curso, destacando sua motivação, os problemas investigados e os principais objetivos.

1.1 Contextualização

A era digital tem sido marcada por uma crescente geração de dados provenientes de diversas fontes, como redes sociais, dispositivos móveis, sensores de Internet of Things (IoT) e sistemas transacionais. Nesse cenário, o conceito de *Big Data* emerge como um paradigma essencial para lidar com essa grande quantidade de informações de maneira eficaz, possibilitando análises complexas e decisões baseadas em dados. Com o aumento da necessidade de extração de valor a partir desses dados, surgem arquiteturas e processos especializados como os *pipelines* de dados, que são fluxos estruturados que extraem, transformam e carregam dados para ambientes de análise e decisão.

Historicamente, o ecossistema de dados nas organizações foi dominado pelas ferramentas proprietárias, ou seja, as fornecidas por grandes empresas, como Amazon, Microsoft e Google Septian et al. [2019]; Dash and Swayamsiddha [2022]. Estas soluções, juntamente com outras proprietárias, são explicitamente destacadas em comparação com alternativas de código aberto Septian et al. [2019]; Dash and Swayamsiddha [2022]. Essas soluções apresentam algumas limitações importantes, dentre elas os altos custos de licenciamento, dependência de fornecedores e restrições de customização. A precificação de muitas plataformas de dados em nuvem, especialmente as proprietárias, adota um modelo baseado no consumo, como o *pay-per-use* ou custos por hora e nó de processamento Murarka et al. [2024]. Este modelo de cobrança por uso e por escalabilidade automática para acomodar *bursty workloads* Murarka et al. [2024], embora ofereça flexibilidade, introduz o custo de implementação e manutenção como um risco significativo no Big Data Septian et al. [2019]. Além disso, a adoção dessas ferramentas pode se tornar um obstáculo para inovação e agilidade operacional, especialmente em contextos onde há necessidade de adaptação rápida a novas demandas de negócio ou escalabilidade. Em resposta

à complexidade e ao risco de custos elevados inerentes aos modelos de precificação de plataformas proprietárias Septian et al. [2019], muitas organizações estão reorientando suas estratégias. Há uma busca crescente por alternativas que sejam mais acessíveis, modulares e flexíveis, sendo as soluções *open source* destacadas como essenciais para permitir que até Pequenas e Médias Empresas (PMEs) enfrentem seus desafios de *Big Data* Septian et al. [2019]; Murarka et al. [2024].

Para viabilizar as operações de dados de forma escalável e segura, e de maneira economicamente sustentável, especialmente em um cenário de constante crescimento do volume de dados, as ferramentas de código aberto *open source* passaram a desempenhar um papel estratégico Murarka et al. [2024]; Septian et al. [2019]. Plataformas como Hadoop, com seus componentes HDFS e MapReduce, tornaram-se essenciais no gerenciamento de conjuntos de dados em larga escala, provendo armazenamento distribuído e capacidades de processamento paralelo cruciais para a escalabilidade Murarka et al. [2024].

Essas ferramentas oferecem alternativas robustas e acessíveis às soluções proprietárias, permitindo que organizações de diferentes portes construam e personalizem suas próprias arquiteturas de dados. Além disso, a natureza colaborativa do *software open source* impulsiona a inovação contínua e facilita a integração com diferentes tecnologias, tornando-se um pilar fundamental na construção de *pipelines* de dados modernos Murarka et al. [2024]; Septian et al. [2019]. O uso de ferramentas como Apache Spark, por exemplo, oferece flexibilidade para acomodar tanto o processamento em lote quanto a manipulação de dados em tempo real, integrando-se de maneira robusta ao ecossistema de *Big Data* Murarka et al. [2024].

Embora o uso de *pipelines* de dados seja essencial para lidar com os desafios do *Big Data* Murarka et al. [2024], muitas organizações enfrentam entraves significativos em sua implementação prática. Entre os principais desafios estão a complexidade da ingestão de dados em tempo real, a integração de fontes heterogêneas e a dificuldade em manter processos de transformação consistentes e escaláveis Murarka et al. [2024]; Nwokeji and Matovu [2021]. Em particular, o ETL (Extract, Transform, Load) tradicional pode ser um processo de custo intensivo e é fundamentalmente uma abordagem unidirecional, não sendo capaz de movimentar dados para fora do *data warehouse* Nwokeji and Matovu [2021]; Dash and Swayamsiddha [2022].

Além disso, há uma ausência de diretrizes padronizadas que auxiliem na seleção, combinação e orquestração de ferramentas *open source* de forma segura e eficiente. A falta de uma abordagem padronizada para modelar os processos e fluxos de trabalho do ETL é reconhecida

como um desafio prevalente Nwokeji and Matovu [2021]. Essa carência resulta na desorientação na escolha e na dificuldade de selecionar um ETL que esteja alinhado às necessidades específicas, preferências e interesses de cada organização Boulahia et al. [2024]. Isso gera incertezas, retrabalho e adoções tecnológicas mal planejadas, resultando em arquiteturas pouco escaláveis ou difíceis de manter.

Diante desse cenário e dos problemas descritos, surge a questão de pesquisa deste trabalho: como ferramentas *open source* podem ser utilizadas na construção de *pipelines* de dados eficientes, escaláveis e acessíveis, e quais são seus impactos em comparação com soluções proprietárias. Visando auxiliar na resposta desta pergunta, este trabalho tem como principal objetivo analisar as ferramentas disponíveis para geração de um *pipeline* de dados funcional e robusto, utilizando exclusivamente tecnologias livres, que seja equivalente aos *pipelines* que são criados utilizando soluções proprietárias. Estudo que vai desde a coleta de dados brutos, realizando *web scraping* de passagens aéreas de sites internacionais com destino ao Brasil, até a disponibilização para análise.

1.2 Objetivos do Trabalho

Este trabalho busca demonstrar a viabilidade técnica e econômica de *pipelines* de dados baseados em ferramentas *open source*, oferecendo uma alternativa às soluções proprietárias tradicionais. O foco recai sobre cenários onde orçamentos limitados e a necessidade de flexibilidade são críticos, tais como pequenas e médias empresas ou projetos acadêmicos. Para isso, é destacado:

1.3 Justificativas da Abordagem *Open Source* no *Pipeline* de Dados

A escolha pela construção de um *pipeline* de dados baseado em tecnologias *open source* visa explorar e demonstrar as seguintes vantagens estratégicas, essenciais para o seu potencial de comparação com soluções proprietárias:

- **Sustentabilidade Econômica e Previsibilidade de Custos:** A abordagem *open source* é fundamental para a viabilidade econômica do *Big Data*, pois evita os custos de licença por usuário ou por volume de dados processados, característicos das plataformas proprietárias Septian et al. [2019]. Esta escolha permite a construção de uma solução economicamente sustentável, com um custo de implementação e manutenção mais previsível, mesmo em

cenários de alta variabilidade de carga Murarka et al. [2024]; Septian et al. [2019].

- **Flexibilidade e Interoperabilidade Arquitetural:** As ferramentas de código aberto não impõem um ecossistema fechado, oferecendo a liberdade e a flexibilidade para escolher e combinar os melhores componentes para cada necessidade específica Murarka et al. [2024]. Esta flexibilidade se manifesta na interoperabilidade entre diferentes tecnologias e se torna um pilar fundamental na construção de *pipelines* de dados modernos Murarka et al. [2024]; Septian et al. [2019].
- **Escalabilidade Técnica Comprovada:** O uso de componentes *open source* é uma estratégia chave para viabilizar operações de dados de forma escalável e segura Murarka et al. [2024]; Septian et al. [2019]. Plataformas *open source*, como o Hadoop e o Apache Spark, demonstraram ser essenciais no gerenciamento de conjuntos de dados em larga escala, provendo armazenamento distribuído e capacidade de processamento paralelo, cruciais para a escalabilidade horizontal da infraestrutura Murarka et al. [2024].
- **Comparação de Desempenho e Viabilidade:** O objetivo final do projeto reside na capacidade de construir e, posteriormente, comparar esta arquitetura *open source* com os modelos proprietários. Essa comparação é justificada pela necessidade de avaliar se as soluções *open source*, que são intrinsecamente mais acessíveis e modulares, podem ser uma alternativa viável e eficiente frente aos desafios de complexidade, volume e heterogeneidade de dados enfrentados pelas organizações Boulahia et al. [2024]; Nwokeji and Matovu [2021].

1.3.1 Objetivos Específicos

1. **Mapear e Catalogar Ferramentas *Open Source*:** Identificar e catalogar as ferramentas de código aberto mais relevantes e robustas para cada etapa do *pipeline* de dados (Ingestão, Transformação, Armazenamento e Ativação), baseando-se nos critérios de avaliação encontrados na literatura para garantir a escalabilidade e a interoperabilidade arquitetural da solução Boulahia et al. [2024]; Murarka et al. [2024]; Septian et al. [2019].
2. **Construir o Protótipo Funcional (*Estudo de Caso*):** Construir um protótipo funcional de um *pipeline* de dados utilizando as ferramentas *open source* selecionadas para o Estudo de Caso. O foco será na ingestão e processamento de dados de passagens aéreas (um domínio com dados variados), abrangendo as seguintes sub-etapas:

- Realizar a **Extração de Dados** de fontes públicas, via *web scraping*, empregando essa técnica onde APIs não são disponíveis ou são limitadas, e sempre respeitando os termos de serviço e a legalidade.
- Efetuar a **Transformação dos Dados** brutos para garantir sua qualidade, consistência e adequação analítica, incluindo padronização (conversão de moedas, limpeza de dados nulos ou duplicados) e o enriquecimento com informações contextuais (dados geográficos).
- Concluir a **Carga dos Dados** transformados em um banco de dados otimizado para consultas analíticas, servindo como a camada de consumo para análises e visualizações.

3. **Avaliar o Desempenho Operacional:** Focar na mensuração e análise do desempenho do *pipeline* de dados construído, quantificando sua eficiência e eficácia por meio de métricas-chave:

- Medir o **Tempo Médio de Processamento** por lote de dados, desde o início da ingestão até a carga final, utilizando logs de tempo ou ferramentas de orquestração para determinar o *throughput* do *pipeline* sob diferentes cargas.
- Monitorar o **Consumo de Recursos Computacionais** (CPU e memória) pelos processos, empregando ferramentas de monitoramento do sistema para identificar gargalos e otimizar a *performance*.
- Estimar os **Custos Operacionais**, calculando os gastos com a infraestrutura (servidores, armazenamento, transferência de dados) da operação do *pipeline*, mesmo com o uso de ferramentas *open source* sem custo de licenciamento.

4. **Realizar a Análise Crítica e Documentar Lições Aprendidas:** Documentar os desafios, as lições aprendidas e oferecer *insights* para futuras implementações, buscando contextualizar os achados:

- Analisar as **Limitações de Escalabilidade** das ferramentas *open source* em cenários de alta escala, comparando o desempenho do protótipo com *benchmarks* de soluções proprietárias ou com as expectativas teóricas para volumes de dados muito maiores (escala petabyte).

- Documentar **Estratégias para Mitigar Vendor Lock-in**, identificando dependências inadvertidas em ecossistemas ou tecnologias específicas, apesar de a abordagem *open source* ser vista como o meio principal de evitar essa restrição.

1.4 Metodologia de Pesquisa: Design Science Research (DSR)

Este trabalho adota o paradigma *Design Science Research* (DSR), uma abordagem metodológica fundamental para a pesquisa em Sistemas de Informação que se foca na criação de artefatos inovadores e úteis para solucionar problemas práticos, conforme proposto por Peffers et al. [2012]. A DSR é a abordagem mais apropriada, pois alinha a necessidade de contribuição acadêmica (geração de conhecimento sobre a viabilidade de soluções livres) com a necessidade prática (construção de um protótipo funcional).

O artefato central deste TCC é a **Arquitetura de Pipeline de Dados Open Source para Big Data** e a documentação detalhada da sua avaliação de desempenho. O artefato compreende um pipeline end-to-end que abrange web scraping (Selenium), armazenamento analítico (DuckDB/PostgreSQL), transformação (dbt), orquestração (Airflow) e visualização (Apache Superset). O processo metodológico é estruturado em seis etapas interligadas, mapeando os objetivos específicos do projeto às fases canônicas da DSR.

1.4.1 Fluxo Metodológico (DSR)

O processo de pesquisa e desenvolvimento é concebido como um fluxo de trabalho sequencial. A Figura 1 ilustra este fluxo, alinhando as etapas do projeto (**M1-M6**) com as fases do *Design Science Research* (DSR) Peffers et al. [2012].

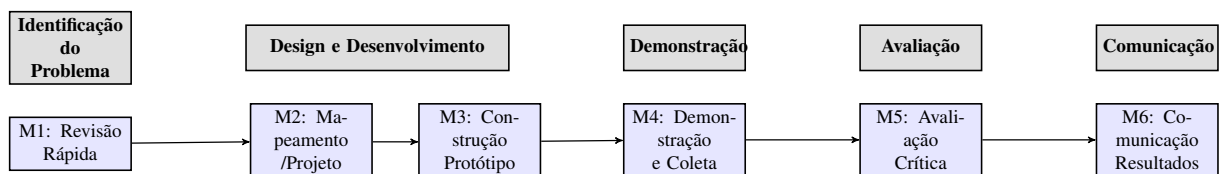


Figura 1: Fluxo Metodológico do Projeto Mapeado às Fases do *Design Science Research* (DSR)

A Tabela 1 detalha as atividades, suas respectivas fases DSR e os resultados concretos esperados.

ID	Atividade	Fase DSR	Resultado (Artefato) e Descrição
M1	Revisão Rápida da Literatura	Identificação do Problema e Objetivos	Catálogo de Ferramentas <i>Open Source</i>: Base teórica de tecnologias e Critérios de Avaliação de ETL Boulahia et al. [2024] para guiar a seleção (Capítulo 3).
M2	Mapeamento e Projeto da Arquitetura	Design e Desenvolvimento	Arquitetura Conceitual e Lógica: Definição formal do <i>stack</i> tecnológico (ferramentas específicas) e do fluxo de dados que será implementado no protótipo.
M3	Construção do Protótipo Funcional	Design e Desenvolvimento	Protótipo do Pipeline Funcionando: Código implementado para Extração (<i>web scraping</i>), scripts de Transformação (limpeza e enriquecimento) e Carga dos dados (Capítulo 4).
M4	Demonstração e Coleta de Métricas	Demonstração	Dados Operacionais Brutos: Logs de Tempo Médio de Processamento e relatórios de Consumo de Recursos Computacionais (CPU/Memória) sob carga, essenciais para a avaliação.
M5	Avaliação Crítica e Análise Comparativa	Avaliação	Análise Crítica e Lições Aprendidas: Documentação dos <i>benchmarks</i> de desempenho, limitações de escalabilidade e a confirmação da viabilidade econômica das soluções <i>open source</i> (Capítulo 5).
M6	Comunicação dos Resultados	Comunicação	Trabalho de Conclusão de Curso (TCC): Documento final que formaliza o artefato criado e o novo conhecimento gerado.

Tabela 1: Etapas Metodológicas, Resultados e Mapeamento DSR

1.5 Organização do texto

Este trabalho de conclusão de curso está organizado em cinco capítulos, seguindo o fluxo metodológico detalhado na Seção 1.4 e na Tabela 1. Neste capítulo, foi apresentado o contexto em que ele se insere, sua motivação e a questão de pesquisa.

O Capítulo 2 apresenta a fundamentação teórica de conceitos que são essenciais para o entendimento dos demais capítulos escritos, incluindo a definição de *Big Data*, a evolução dos *pipelines* de dados (ETL, ELT, Reverse ETL), e o detalhamento das ferramentas *open source*

mais relevantes.

O Capítulo 3, correspondente à **Etapa M1** (Revisão Rápida da Literatura), apresenta uma revisão diante dos materiais encontrados em uma *Rapid Review* Dobbins [2017] sobre o tema. Este capítulo demonstra as características essenciais para a seleção de ferramentas, conforme os critérios de avaliação propostos por Boulahia et al. Boulahia et al. [2024], fornecendo o embasamento teórico para a **Etapa M2** de projeto.

O Capítulo 4, correspondente às **Etapas M2, M3 e M4** da metodologia, apresenta a proposta de solução do problema e a demonstração prática do artefato. O foco reside na construção e teste de um *pipeline* de dados robusto, desde a coleta via *web scraping* de passagens aéreas até a disponibilização dos dados para análise, demonstrando que é possível atingir resultados comparáveis aos de soluções comerciais.

O Capítulo 5, correspondente à **Etapa M5** (Avaliação Crítica e Análise Comparativa) e **M6** (Comunicação), apresenta uma discussão final sobre o conteúdo que foi apresentado nesse trabalho de pesquisa, analisando os resultados da performance, suas contribuições e limitações.

Capítulo 2

Fundamentação Teórica

Neste capítulo, são apresentados os principais conceitos e estudos que embasam o desenvolvimento de *pipelines* de dados com ferramentas *open source*. A seguir, aborda-se a evolução das abordagens de ETL (*Extract, Transform, Load*), os desafios contemporâneos da engenharia de dados e os benefícios da adoção de soluções de código aberto.

2.1 ETL e a Evolução das Arquiteturas de Dados

O processo de *Extract, Transform, Load* (ETL) consolidou-se como o componente fundamental na engenharia de dados, responsável por extrair informações de diversas fontes, transformá-las conforme necessidades analíticas e carregá-las em ambientes como *data warehouses* (DW) ou *data lakes* (DL). O DW é uma estrutura de banco de dados desnormalizado e centralizado, projetada para fornecer suporte à tomada de decisão estratégica por meio de análise histórica, amplamente utilizada em operações de *Online Analytical Processing* (OLAP) e *data mining* (Septian et al. [2019]).

Em contrapartida, o *data lake* representa uma abordagem eficiente e escalável para o armazenamento de grandes volumes de dados brutos em diversos formatos, permitindo que o processamento ocorra sob demanda. Com suporte a ferramentas *open source*, como o Hadoop File System, os *data lakes* oferecem vantagens como baixo custo, tolerância a falhas e flexibilidade de esquema (*schema on read*).

Essa evolução arquitetural é um reflexo direto dos desafios impostos pelo *Big Data*. Segundo Nwokeji and Matovu [2021], o ETL evoluiu para lidar com os cinco principais desafios do Big Data, conhecidos como os 5Vs: volume, variedade, velocidade, veracidade e valor. No entanto, processos tradicionais de ETL, tipicamente associados ao contexto de *Business Intelligence* (BI), frequentemente demonstram limitações em ambientes modernos, incluindo custo elevado e ingestão limitada em tempo real (Nwokeji and Matovu [2021]).

Para superar essas restrições, houve uma evolução significativa nos modelos. Boulahia et al. [2024] categorizam o ETL em três grandes vertentes: ETL para *Business Intelligence* (tradi-

cional, focado em dados estruturados), ETL para *Big Data* (utilizando tecnologias distribuídas como Hadoop e Spark para escalabilidade) e ETL Semântico (que utiliza ontologias para integração de dados heterogêneos).

Em paralelo, a própria ordem do processamento foi redefinida. Surgiu a abordagem ELT (*Extract, Load, Transform*), na qual os dados brutos são primeiramente carregados no *data lake* ou *data warehouse* e a transformação ocorre posteriormente, explorando o poder de processamento do próprio destino. Essa flexibilidade é crucial para o desenvolvimento de *pipelines* híbridos e escaláveis.

Transição para ELT e o Papel do Data Wrangling

Um desafio central que afeta a escolha é a **complexidade** do *Data Wrangling* (obter, organizar, entender, extrair e formatar dados). A complexidade e a escala do *Big Data* impulsionaram a transição do modelo tradicional **ETL** (*Extract, Transform, Load*) para o **ELT** (*Extract, Load, Transform*). No modelo ELT, a transformação ocorre *após* o carregamento dos dados brutos no *Data Lake*, aproveitando a capacidade de processamento distribuído da plataforma *open source*.

Apesar do desafio de *setup* e manutenção (*steep learning curve*), o modelo *open source*, exemplificado pelo Apache Airflow, é a opção preferida por sua escalabilidade, flexibilidade e a capacidade de ser um "contêiner aberto" para diversas soluções de *data wrangling*.

2.2 Open Source: Estratégia, Flexibilidade e Comparação

O uso de ferramentas *open source* vem se consolidando como alternativa viável e estratégica às soluções proprietárias em projetos de engenharia de dados. Septian et al. [2019] demonstram que, ao construir *pipelines* com softwares livres, é possível reduzir custos, adaptar a arquitetura às necessidades específicas da organização e fomentar a inovação por meio da colaboração comunitária.

Especificamente para pequenas e médias empresas, Septian et al. [2019] apontam que soluções baseadas em software livre oferecem alternativas de baixo custo para acessar tecnologias antes restritas a grandes corporações. Ferramentas como Apache Hadoop, Spark e Kafka são amplamente utilizadas em diversos ambientes corporativos, viabilizando a escalabilidade e a interoperabilidade entre diferentes camadas da arquitetura.

A agilidade na evolução tecnológica é outro fator decisivo. Murarka et al. [2024] destacam que a natureza colaborativa das comunidades *open source* impulsiona a inovação contínua,

criando ecossistemas que evoluem rapidamente. Essa característica contrasta com o ciclo mais lento de atualização das ferramentas proprietárias.

A escolha entre as abordagens, no entanto, depende do contexto organizacional. Boulahia et al. [2024] apresentam uma análise comparativa:

- Ferramentas Proprietárias: Oferecem maior robustez, suporte técnico especializado e garantias contratuais.
- Ferramentas *Open Source*: Destacam-se pela flexibilidade, personalização e baixo custo.

Desta forma, ambientes acadêmicos e empresas com foco em inovação e adaptabilidade podem beneficiar-se das soluções *open source*, enquanto grandes corporações com alta criticidade de dados e requisitos regulatórios podem ainda optar por ferramentas proprietárias (Boulahia et al. [2024], Septian et al. [2019]).

2.3 Técnicas Avançadas de Ingestão e Processamento

Com o aumento exponencial da produção de dados provenientes de dispositivos *IoT*, redes sociais e sistemas transacionais, surgem novos desafios na ingestão em tempo real. Murarka et al. [2024] enfatizam que tecnologias como Apache Kafka, Flume e NiFi desempenham papel central nesse contexto, oferecendo suporte eficiente tanto a fluxos *batch* quanto *streaming*, sendo alternativas ao modelo tradicional do Hadoop MapReduce por oferecerem processamento em memória e análises em tempo real.

A capacidade de lidar com diferentes padrões de fluxo de dados é crucial. Plataformas como Apache Spark, Flink e Storm permitem a construção de *pipelines* mais eficientes, com menor latência e maior resiliência.

Um aspecto relevante, e tendência crescente, é a incorporação de aprendizado de máquina (*machine learning*) nos *pipelines* de dados. O objetivo é automatizar etapas como detecção de anomalias, classificação e priorização de fluxos. Essa integração aumenta a eficiência operacional e abre espaço para *pipelines* mais autônomos e adaptativos (Murarka et al. [2024]).

2.4 Tendências Emergentes: Reverse ETL

Tradicionalmente, a arquitetura analítica de dados foca na consolidação de dados em repositórios centrais. No entanto, o conceito de *Reverse ETL* (*Extract, Transform, Load* Invertido) tem gan-

hado destaque ao propor o fluxo de dados no sentido oposto: devolver os dados já tratados, enriquecidos e analisados a sistemas operacionais e de negócio.

Dash and Swayamsiddha [2022] explicam que essa abordagem resolve o chamado “último quilômetro da análise de dados”, reduzindo silos de informação e permitindo que *insights* estratégicos sejam consumidos diretamente em ferramentas operacionais (como CRMs, ferramentas de marketing ou ERPs). Isso alinha decisões analíticas com ações operacionais em tempo quase real.

Apesar de ser uma tendência poderosa para a *Operational Analytics*, o *Reverse ETL* ainda enfrenta desafios técnicos como a sincronização eficiente de dados, a integração com múltiplas APIs de terceiros e a gestão de custos associados à latência, temas que continuam a impulsionar o desenvolvimento de novas ferramentas no ecossistema de dados (Dash and Swayamsiddha [2022]).

Capítulo 3

Revisão da Literatura

O método de pesquisa em questão utiliza a **Revisão Rápida (*Rapid Review*)**, uma abordagem que se assemelha à **Revisão Sistemática (*Systematic Review*)**, mas que é conduzida de forma a ser mais oportuna e com custos reduzidos Pena et al. [2025].

A Revisão Sistemática é um estudo secundário rigoroso e abrangente, projetado para localizar, avaliar criticamente e sintetizar todas as evidências relevantes sobre uma questão de pesquisa claramente definida, minimizando o viés e gerando conclusões confiáveis e cientificamente sólidas.

Em contraste, a Revisão Rápida adapta o método da Revisão Sistemática para entregar evidências em prazos reduzidos e a custos menores, geralmente simplificando procedimentos como a limitação das fontes de busca, o uso de apenas um avaliador para a triagem de estudos ou a omissão de uma avaliação de qualidade detalhada Pena et al. [2025]. Dada a necessidade de agilidade na coleta e análise de literatura para o trabalho em questão, opta-se pela metodologia de Revisão Rápida para a condução deste estudo.

Para garantir rigor metodológico na seleção das fontes e na construção deste trabalho, foi adotado um rigoroso protocolo de revisão baseado no modelo *PICOC* (*Population, Intervention, Comparison, Outcome, Context*). Esse modelo permitiu estruturar a busca de forma sistemática, relacionando diretamente os elementos da investigação com os objetivos do trabalho. Os elementos definidos foram:

- **Population:** Data engineer, Data processing, Big Data, SQL, Data Lake e Data Warehouse.
- **Intervention:** ETL, ELT e Extract Transform Load.
- **Comparison:** N/A.
- **Outcome:** N/A.
- **Context:** N/A.

Com base nesse enquadramento, foram definidas as seguintes questões de pesquisa (*Research Questions*):

1. Quais são as principais ferramentas open source utilizadas no desenvolvimento de pipelines de dados atualmente?
2. De que forma o uso de ferramentas open source impacta a eficiência, flexibilidade e custo dos pipelines de dados?
3. Em quais contextos as ferramentas open source são mais vantajosas para o desenvolvimento de pipelines de dados?
4. Quais os benefícios e limitações do uso de ferramentas open source em comparação com ferramentas proprietárias no contexto da engenharia de dados?

Para operacionalizar a busca, foi utilizada a seguinte *search string* em bases de dados científicas:

```
TITLE-ABS-KEY ( ( "data engineer*"OR "data processing"OR "big data*"OR
sql OR "Data lake"OR datawarehouse ) AND ( etl OR elt OR "extract
transform load" ) ) AND PUBYEAR > 2019 AND PUBYEAR < 2026 AND ( LIMIT-TO
( SUBJAREA , "COMP") OR LIMIT-TO ( SUBJAREA , "ENGI" ) )
```

A execução dessa estratégia de busca retornou inicialmente um total de **315 publicações**. Para organizar o processo de triagem, foi utilizada a ferramenta **Parsifal**, amplamente empregada em revisões sistemáticas da literatura. Esses trabalhos constituem a base para a análise de avanços recentes, comparação de ferramentas e identificação de tendências emergentes no campo dos pipelines de dados.

A escassez de artigos diretamente identificados pela string de busca inicial, um desafio comum em revisões bibliográficas de escopo restrito ou emergente, levou a uma busca de literatura ad hoc complementar. Esta abordagem pragmática, porém sistemática, permitiu a inclusão de informações cruciais e altamente relevantes de um conjunto mais amplo de fontes, garantindo a robustez necessária para responder de forma completa e fundamentada às questões de pesquisa propostas.

3.0.1 Artigos Selecionados para a Revisão

A execução dessa estratégia de busca retornou inicialmente um total de **315 publicações**. Para organizar o processo de triagem, foi utilizada a ferramenta **Parsifal**, amplamente empregada em revisões sistemáticas da literatura. Após a leitura dos resumos, aplicação de critérios de inclusão e exclusão e a verificação de alinhamento com as *Research Questions*, apenas **5 artigos foram selecionados** para compor a presente revisão. Esses trabalhos constituem a base para a análise de avanços recentes, comparação de ferramentas e identificação de tendências emergentes no campo dos pipelines de dados.

Citação	Título e Foco Principal	Relevância para o Projeto
Boulahia et al. [2024]	<i>The multi-criteria evaluation of research efforts based on ETL software</i> (Avaliação de múltiplos critérios (AHP) para escolha de ETLs em BI, Big Data e contexto semântico.)	Critérios de avaliação de ETLs Open Source (M1).
Nwokeji and Matovu [2021]	<i>A Systematic Literature Review on Big Data Extraction, Transformation and Loading (ETL)</i> (Revisão sistemática sobre ETL para Big Data, abordando abordagens e atributos de qualidade.)	Estrutura e atributos de qualidade para ETL em Big Data (M1).
Murarka et al. [2024]	<i>Advanced Techniques in Data Ingestion and Pipelining for Scalable Big Data Platforms</i> (Técnicas avançadas de ingestão e processamento para Big Data escalável, com uso de Kafka, Spark e ML/AI.)	Demonstra a escalabilidade de ferramentas como Kafka e Spark no ecossistema Open Source (M2/M5).
Septian et al. [2019]	<i>A Study Review of Common Big Data Architecture for Small-medium Enterprise</i> (Arquitetura de Big Data e pipelines para Pequenas e Médias Empresas, com foco em Open Source.)	Justifica o foco em Open Source para PMEs (M1) e serve de base para a arquitetura (M2).
Dash and Swayamsiddha [2022]	<i>Reverse ETL for Improved Scalability, Observability, and Performance of Modern Operational Analytics – A Comparative Review</i> (Análise e importância do Reverse ETL para o fluxo inverso do Data Warehouse para sistemas operacionais.)	Identifica a tendência emergente do Reverse ETL, essencial para a análise crítica (M5).

Tabela 2: Artigos Selecionados para a Revisão Rápida

3.1 Respostas às Questões de Pesquisa

A análise da literatura selecionada, complementada pela busca ad hoc, permite responder às questões de pesquisa propostas, traçando um panorama atualizado sobre as ferramentas open source, suas vantagens, desvantagens e o contexto de aplicação em pipelines de dados.

3.1.1 Quais são as principais ferramentas open source utilizadas no desenvolvimento de pipelines de dados atualmente?

A revisão da literatura aponta para um ecossistema de pipelines de dados dominado por ferramentas open source, especialmente no contexto de plataformas de *Big Data* e para Pequenas e Médias Empresas (PMEs) Septian et al. [2019]; Murarka et al. [2024]; Mbata et al. [2024]. As principais ferramentas de código aberto são categorizadas de acordo com a fase do pipeline de dados em que são aplicadas:

1. **Ingestão de Dados e Filas de Mensagens (*Data Ingestion and Message Queues*):** O **Apache Kafka** é amplamente reconhecido como a principal ferramenta *open source* para processamento de *streams* de dados em tempo real, fornecendo escalabilidade e tolerância a falhas na ingestão de dados de alta velocidade Murarka et al. [2024]; Mbata et al. [2024]. Ferramentas como **Apache Flume** e **Apache Nifi** são citadas para ingestão em lote (*batch*) e em *stream* de dados de diversas fontes, como *logs* e sistemas baseados em eventos Murarka et al. [2024]. O **RabbitMQ** também é mencionado como outra opção popular de fila de mensagens *open source* Septian et al. [2019].
2. **Processamento, Transformação e Orquestração (ETL/ELT):** O **Apache Spark** é a ferramenta de processamento de dados mais destacada, considerada uma alternativa robusta ao tradicional MapReduce. O Spark oferece processamento em memória, acelerando drasticamente as velocidades de processamento de dados e suportando tanto o processamento em lote quanto em tempo real Murarka et al. [2024]; Septian et al. [2019]; Mbata et al. [2024]. Para orquestração e gerenciamento de fluxo de trabalho:
 - **Apache Airflow:** Utilizado para orquestração e gerenciamento de fluxo de trabalho (*Workflow Management*) e *pipeline* de *Machine Learning* (ML) Mbata et al. [2024].
 - **Apache NiFi:** Mencionado para a criação de fluxos de trabalho ETL, ingestão e transformação de dados, além de gerenciar fluxos de dados em tempo real Mbata et al. [2024].
3. **Armazenamento (*Data Lake e Data Warehouse*):** O **Hadoop Distributed File System (HDFS)** é citado como o componente fundamental *open source* para a construção de

um *Data Lake*, permitindo o armazenamento confiável e o processamento distribuído de grandes conjuntos de dados Murarka et al. [2024]; Septian et al. [2019].

4. **Visualização e *Business Intelligence* (BI):** Produtos como **Apache Zeppelin** e **Metabase** são citados como exemplos de ferramentas de código aberto que auxiliam na visualização dos *insights* e podem se conectar com sistemas como HDFS e diversos bancos de dados Septian et al. [2019].

3.1.2 De que forma o uso de ferramentas open source impacta a eficiência, flexibilidade e custo dos pipelines de dados?

O impacto das ferramentas *open source* nos *pipelines* de dados é positivo em três dimensões críticas: custo, flexibilidade e eficiência, sendo particularmente relevante para empresas de menor porte Septian et al. [2019].

- **Custo (Custo-Benefício):** Para Pequenas e Médias Empresas (PMEs), a adoção de ferramentas *open source* é frequentemente justificada pela **redução de custos de licenciamento de software** (*low cost over a software license*). O uso de produtos como Apache Hadoop, Kafka e Spark permite que as empresas iniciem seus projetos de *Big Data* com um custo mínimo de licença para a implementação inicial (*minimum cost over a software license for the early implementation*).
- **Flexibilidade (*Flexibility*):** A flexibilidade é uma das qualidades mais valorizadas em soluções ETL/ELT e está intrinsecamente ligada à capacidade de uma solução acomodar **mudanças nos requisitos de integração de dados** e adaptar-se a volumes e complexidades de dados variáveis.
- **Adaptabilidade Tecnológica:** Ferramentas como Apache Spark e Apache Kafka oferecem **integração contínua e perfeita com o ecossistema de *Big Data***, permitindo que as organizações construam *multi-service data pipelines* complexos com relativa facilidade.
- **Reutilização (*Reusability*):** O atributo de *reusability* define a capacidade de uma solução ETL/ELT ser usada em **vários domínios de aplicação** (*various application domains*), uma característica que o código aberto facilita por sua natureza não restritiva e ampla compatibilidade.

- **Eficiência (*Performance e Escalabilidade*):** A eficiência dos *pipelines* é frequentemente medida pelo atributo de *performance*, que exige que o processo ETL seja concluído em uma **janela de tempo especificada**.
- **Processamento Distribuído:** Plataformas *open source* como o Hadoop com HDFS e o Apache Spark são essenciais para a eficiência em escala, pois fornecem capacidades de armazenamento distribuído e processamento paralelo, acelerando drasticamente o processamento de grandes volumes de dados.
- **Escalabilidade (*Scalability*):** A escalabilidade refere-se à capacidade de uma solução ETL/ELT ser usada para volumes e complexidades de dados variáveis. O design inerente de ferramentas como Kafka (mensageria distribuída) e Spark (processamento em memória) as torna soluções robustas para gerenciar *Big Data* em escala.

3.1.3 Em quais contextos as ferramentas open source são mais vantajosas para o desenvolvimento de pipelines de dados?

As ferramentas *open source* demonstram ser vantajosas em diversos contextos, sendo o fator predominante a necessidade de escalabilidade e flexibilidade em ambientes de *Big Data* e a otimização de custos em organizações com recursos limitados Septian et al. [2019]; Murarka et al. [2024].

- **Contexto de *Big Data* e *Scalability*:** A vantagem do código aberto é máxima em projetos que lidam com as características V's do *Big Data* (Volume, Velocidade, Variedade) Septian et al. [2019]. Ferramentas como Apache Spark e Apache Kafka são fundamentais em cenários que exigem análise imediata e baixa latência Murarka et al. [2024]. O uso do Spark (processamento em memória) oferece grande vantagem sobre modelos tradicionais em lote para cenários que necessitam de agilidade e escalabilidade horizontal Murarka et al. [2024].
- **Contexto de PMEs e Custo (*Cost-effective*):** Para pequenas e médias empresas, o código aberto é a principal via de acesso à infraestrutura de *Big Data* Septian et al. [2019]. A ausência de custos de licença (*low cost over a software license*) permite que as PMEs experimentem e cresçam com a infraestrutura sem a barreira financeira associada a soluções proprietárias de grande escala Septian et al. [2019].

- **Contexto de Inovação e *Feature Engineering* (ML/AI):** A vasta e criativa comunidade *open source* (*vibrant and creative open-source ecosystem*) é o "combustível para foguetes" (*rocket fuel*) para pesquisa e inovação em *Machine Learning* (ML) e Inteligência Artificial (AI) Reddi et al. [2021]. Em contextos de engenharia de dados voltados para alimentar modelos de ML, a colaboração em torno de conjuntos de dados e ferramentas *open source* acelera o desenvolvimento de novos modelos e a reprodutibilidade dos resultados Reddi et al. [2021].

3.1.4 Quais os benefícios e limitações do uso de ferramentas open source em comparação com ferramentas proprietárias no contexto da engenharia de dados?

A comparação entre ferramentas *open source* e proprietárias é um tema central na Engenharia de Dados. A literatura fornece uma visão clara sobre os *trade-offs* envolvidos, especialmente em termos de custo, suporte e complexidade operacional Septian et al. [2019]; Nwokeji and Matovu [2021]; Murarka et al. [2024]; Mbata et al. [2024].

Benefícios do *Open Source* em Comparação com o Proprietário

Benefício	Descrição	Fontes
Custo Inicial Reduzido	Eliminação ou redução drástica dos custos de licença, o que é crucial para a implementação inicial por PMEs Septian et al. [2019].	Septian et al. [2019]
Flexibilidade e Reutilização	Maior adaptabilidade às mudanças de requisitos e capacidade de ser reutilizada em múltiplos domínios Nwokeji and Matovu [2021].	Nwokeji and Matovu [2021]
Inovação Acelerada (ML/AI)	Acesso imediato ao ecossistema global de inovação, impulsionando a pesquisa e o desenvolvimento de modelos de <i>Machine Learning</i> Reddi et al. [2021].	Reddi et al. [2021]

Tabela 3: Benefícios do *Open Source* em *Data Pipelines*

Limitações do *Open Source* em Comparação com o Proprietário

Limitação		Descrição	Fontes
Suporte e Manutenção		O suporte tende a ser baseado na comunidade, podendo levar a uma curva de aprendizado mais íngreme e custos indiretos com a complexidade de <i>setup</i> e monitoramento operacional Mbata et al. [2024]; Septian et al. [2019].	Mbata et al. [2024]; Septian et al. [2019]
Complexidade Operacional		A configuração e a manutenção podem ser complexas e exigir um alto nível de especialização técnica. Soluções proprietárias baseadas em nuvem oferecem serviços gerenciados, simplificando essa complexidade Septian et al. [2019]; Murarka et al. [2024].	Septian et al. [2019]; Murarka et al. [2024]
Observabilidade (Reverse ETL)		O <i>Reverse ETL</i> é mais difícil devido à necessidade de sincronização manual e reconciliação de dados, um desafio simplificado por ferramentas proprietárias focadas em <i>Operational Analytics</i> Dash and Swayamsiddha [2022].	Dash and Swayamsiddha [2022]
Foco na Ferramenta		Soluções <i>open source</i> , por vezes, carecem de um gerenciamento de fluxo de trabalho de ponta a ponta e componentes "prontos para usar", exigindo a combinação de várias ferramentas para obter uma solução completa Mbata et al. [2024].	Mbata et al. [2024]

Tabela 4: Limitações do *Open Source* em *Data Pipelines*

Capítulo 4

Construção do Pipeline

Para a construção do *pipeline* de dados, foram mapeadas e extraídas informações de dez websites internacionais, de diferentes países, que oferecem pacotes de viagem e passagens aéreas com destino ao Brasil. Essa diversidade de fontes enriquece o conjunto de dados, permitindo uma análise comparativa mais ampla e detalhada. Contudo, essa variedade também impôs desafios significativos, especialmente em relação à multi-moeda e ao multi-idioma. A extração de dados exigiu que os scripts fossem adaptados para lidar com diferentes moedas (Euros, Libras, Dólares americanos e Dólares chilenos) e idiomas (alemão, espanhol, francês e inglês), garantindo que os dados brutos fossem coletados de forma consistente, apesar das variações. Foram desenvolvidos dez scripts para cada site escolhido: Bleu Selectour (França), Comptoir Des Voyages (França), Ikarus Tours (Alemanha), Jetmar (Uruguai), Journey Latin America (Reino Unido), Newmarket Holidays (Reino Unido), Panamericana Turismo (Chile), Sol Férias (Portugal), Transalpino (Portugal) e Turismo Costanera (Chile).

O foco principal do trabalho é a avaliação de ferramentas de código aberto, analisando seu impacto direto na escalabilidade e no desempenho da solução. A metodologia empregada aborda os principais desafios na elaboração da arquitetura, desde a extração de dados dinâmicos até o processamento e armazenamento eficientes.

Em suma, este estudo de caso serve como um guia prático para a construção de soluções de dados eficientes, provando que é possível desenvolver sistemas de alta performance utilizando um ecossistema de ferramentas acessíveis e flexíveis. Além de seu caráter acadêmico e tecnológico, este trabalho também possui relevância prática, podendo servir como base para instituições como a Embratur, que dependem da coleta e análise de informações de mercado internacional para planejar estratégias de promoção turística do Brasil no exterior.

Tecnologias Utilizadas

O processo de construção do pipeline de dados foi integralmente desenvolvido em Python, uma linguagem de programação cuja vasta aplicabilidade em automação e no ecossistema de processamento de dados a torna uma escolha robusta para projetos de Engenharia de Dados.

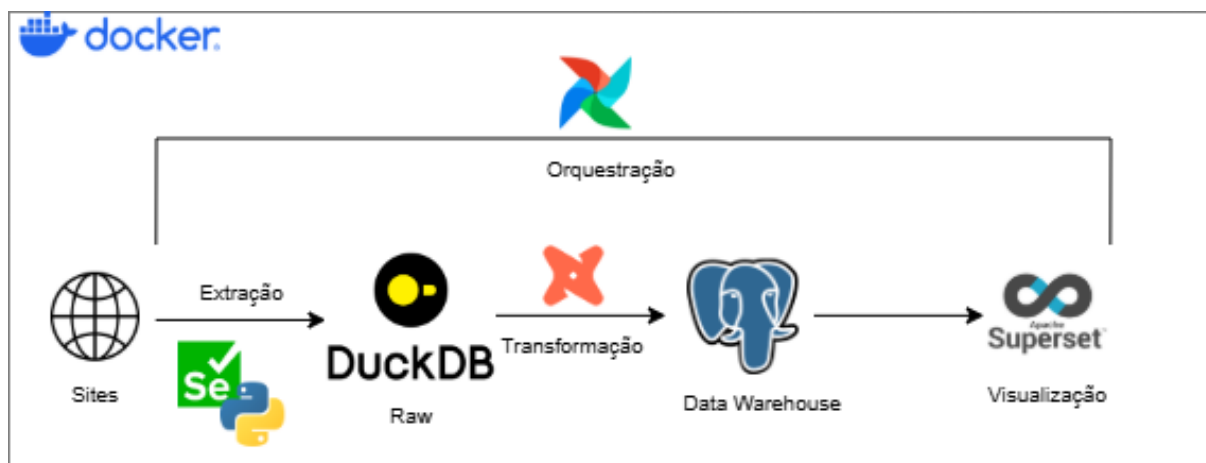


Figura 2: Arquitetura completa do pipeline de dados

A Figura 2 apresenta a visão geral da arquitetura do pipeline desenvolvido. O fluxo de dados percorre cinco etapas: (1) extração dos dados de dez sites internacionais via Selenium e Python; (2) armazenamento dos dados brutos no DuckDB, que atua como repositório temporário (*raw*); (3) transformação e padronização dos dados pelo dbt; (4) consolidação no PostgreSQL, que serve como *data warehouse* analítico; e (5) visualização por meio do Apache Superset. O Apache Airflow orquestra todas as etapas de forma automatizada, e toda a infraestrutura é containerizada via Docker Compose, garantindo a reprodutibilidade do ambiente.

Para a fase crucial de aquisição e extração de conteúdo dinâmico (*web scraping*), a ferramenta central adotada foi o **Selenium WebDriver**. Este framework de automação de navegadores serviu como a espinha dorsal da solução, simulando o comportamento de um usuário real (como cliques, rolagem de página e interações complexas com elementos de interface, utilizando recursos da biblioteca que permitem simular ações do mouse e selecionar opções em menus suspensos). Sua capacidade de lidar com interações complexas garante a extração de dados de fontes que demandam alta flexibilidade de acesso.

Uma vez extraídos, os dados brutos foram imediatamente direcionados para o **DuckDB**, que atuou como repositório temporário de alto desempenho. O DuckDB é um sistema de banco de dados relacional e analítico que se destaca por sua arquitetura embutida, ou seja, ele funciona diretamente dentro do programa que o utiliza, sem a necessidade de instalar ou manter um servidor de banco de dados separado. Essa característica simplifica a arquitetura do projeto e minimiza os custos indiretos associados à complexidade operacional e manutenção de um

componente dedicado.

Após a ingestão inicial, os dados brutos foram migrados para o **PostgreSQL**, um sistema de gerenciamento de banco de dados relacional open source que atua como o *data warehouse* analítico do projeto. Diferentemente do DuckDB, o PostgreSQL opera no modelo cliente-servidor, o que significa que ele funciona como um serviço independente ao qual múltiplas ferramentas podem se conectar simultaneamente, uma característica essencial para a integração com as demais tecnologias do pipeline. Sua robustez, ampla conformidade com padrões SQL e mais de 35 anos de desenvolvimento ativo o consolidam como uma das soluções open source mais confiáveis para cargas analíticas.

Para a camada de transformação dos dados, foi adotado o **dbt** (*data build tool*), uma ferramenta open source que implementa o paradigma de transformação como código (*transformation as code*). Em termos práticos, isso significa que todas as regras de limpeza, padronização e reorganização dos dados são escritas em SQL, a linguagem padrão para consultas em bancos de dados, e armazenadas em arquivos versionáveis, da mesma forma que se versiona o código de um software. O dbt opera exclusivamente sobre os dados já carregados no PostgreSQL, alinhando-se à abordagem ELT discutida no Capítulo 2. Seu sistema integrado de testes automáticos e documentação agrega uma camada de qualidade e governança que seria difícil de replicar com scripts avulsos.

A orquestração de todo o pipeline, desde a execução dos scripts de web scraping até a materialização dos modelos dbt, foi confiada ao **Apache Airflow**, uma plataforma open source de gerenciamento de fluxos de trabalho. O Airflow permite a definição de pipelines como código Python, utilizando o conceito de DAGs (*Directed Acyclic Graphs*), que são grafos direcionados sem ciclos nos quais cada nó representa uma tarefa a ser executada e as setas entre eles definem a ordem de execução, ou seja, qual tarefa precisa terminar antes que a próxima comece. Sua interface web oferece visibilidade completa sobre o histórico de execuções, registros detalhados de cada etapa e mecanismos automáticos de reexecução em caso de falha, garantindo a observabilidade e a resiliência do pipeline em produção.

Por fim, a disponibilização dos dados transformados para análise visual foi realizada por meio do **Apache Superset**, uma plataforma open source de *Business Intelligence* que permite a criação de painéis interativos (*dashboards*) e a exploração de dados via consultas SQL. O Superset conecta-se nativamente ao PostgreSQL e se posiciona como uma alternativa direta a ferramentas proprietárias como Tableau e Power BI, sem custos de licenciamento.

Em paralelo, o processamento e a qualidade dos dados foram aprimorados pelo uso estratégico de bibliotecas nativas do Python. A biblioteca de expressões regulares foi essencial na etapa de transformação para identificar e extrair padrões específicos dentro de textos, como valores monetários e durações de viagem que estavam misturados em frases descritivas. Adicionalmente, bibliotecas auxiliares foram integradas ao pipeline para gerenciar a organização dos arquivos e enriquecer os dados com informações temporais (como a data e hora da coleta), garantindo a rastreabilidade e a consistência do conjunto de dados final.

4.0.1 Tentativa com a ferramenta Browser Use (*Web Scraping* com Inteligência Artificial)

Antes de iniciar a abordagem manual com o Selenium, foi explorada a possibilidade de usar ferramentas de inteligência artificial para a extração de dados. A ferramenta Browser Use, que atua como um agente de navegação autônomo, prometia uma alternativa mais intuitiva e menos dependente de seletores CSS fixos, ao permitir a automação de tarefas online em linguagem natural. Essa abordagem era perfeitamente alinhada com a premissa do trabalho, que busca soluções *open source* e flexíveis. O agente foi configurado para usar o modelo de linguagem Mistral, que rodou localmente via Ollama e foi orquestrado com a biblioteca LangChain.

O Mistral 8x7B é um Grande Modelo de Linguagem (Large Language Model - LLM) notável por sua arquitetura Mixture of Experts (MoE), sendo composto por oito módulos especialistas (*experts*), cada um contendo sete bilhões de parâmetros. Essa estrutura é projetada para especializar-se em diferentes tipos de tarefas e está na vanguarda da pesquisa em aprendizado multimodal, visando aprimorar a integração e o processamento de dados de texto, imagem e áudio.

A tentativa demonstrou que o Browser Use é funcional. O modelo conseguia interagir com as páginas e extrair informações, validando a promessa de que é possível automatizar o processo com comandos de alto nível. No entanto, essa abordagem enfrentou uma limitação técnica significativa: o processamento local do modelo de IA. A execução do Mistral via Ollama se mostrou extremamente intensiva em termos de recursos computacionais, sobrecarregando a máquina utilizada para o projeto.

Os testes foram realizados em um computador DESKTOP-I0H93GR, equipado com um processador AMD Ryzen 7 4800HS with Radeon Graphics (2,90 GHz), 16 GB de memória

RAM (15,4 GB utilizáveis) e sistema operacional Windows 10 de 64 bits. Apesar de possuir boas especificações para tarefas gerais de desenvolvimento, a execução local do modelo exigiu recursos acima da capacidade disponível, resultando em lentidão significativa e, em alguns casos, no desligamento completo do computador devido à alta demanda de CPU e memória RAM.

Essa limitação inviabilizou a continuidade do projeto com essa arquitetura, evidenciando a necessidade de recursos computacionais mais robustos ou da adoção de uma abordagem baseada em infraestrutura em nuvem para o processamento de modelos de linguagem de grande porte.

Essa experiência reforça a importância de considerar o custo operacional e os requisitos de infraestrutura ao escolher as ferramentas para um *pipeline* de dados, especialmente em projetos que buscam a acessibilidade e a eficiência. A tentativa, embora não tenha sido bem-sucedida em termos de implementação prática devido às limitações de *hardware*, serviu como uma valiosa lição sobre as barreiras que ainda existem para o uso de soluções de IA em ambientes de baixo custo, destacando que essa abordagem é promissora para o futuro, desde que haja um *hardware* mais potente à disposição.

4.1 Web Scraping

O *Web Scraping* ou *Web Crawling* refere-se ao procedimento de extração automática de dados de websites utilizando software Khder [2021]. Essa técnica é fundamental em cenários onde os dados não são fornecidos em formatos facilmente legíveis por máquinas, como JSON ou XML, sendo utilizada para extrair dados estruturados a partir de texto, tipicamente em HTML Khder [2021]. Em essência, o *web scraping* é um método para converter dados da web não estruturados em **dados estruturados** que podem ser armazenados e analisados em um banco de dados central ou planilha Khder [2021]; Mbata et al. [2024].

Essa automação é significativamente mais rápida e precisa do que a entrada manual de dados, resultando em informações mais completas, consistentes e acuradas Khder [2021].

4.1.1 Diferenciação e Propósito

Embora os termos *Web Scraping* e *Web Crawling* sejam frequentemente usados de forma intercambiável para descrever a extração automática de dados de websites Khder [2021], é importante notar a distinção. O processo de *web scraping* geralmente envolve a criação e im-

plementação de duas ferramentas de software principais:

1. Um **crawler** (ou *spider*), que tem a função de baixar os dados da Internet de forma sistemática Khder [2021]; Heltweg and Riehle [2023].
2. Um **scraper**, que extrai as informações cruas relevantes do dado baixado, codifica-as e armazena-as em um arquivo ou banco de dados em um formato e estrutura definidos pelo usuário Khder [2021].

O processo de *web scraping* (ou *data acquisition*) é uma das primeiras fases em qualquer estudo ou desenvolvimento de Ciência de Dados, sendo crucial em situações como a Inteligência de Negócios (*Business Intelligence*) e em análises de mercado, onde a coleta de dados é de extrema importância para a tomada de decisões estratégicas Khder [2021]; Reddi et al. [2021]. O uso de *web scraping* tem sido largamente documentado em diversas indústrias, como:

- **Comércio e Varejo:** Para coletar preços em tempo real de sites de varejistas e obter detalhes do produto Khder [2021].
- **Inteligência de Negócios (*Business Intelligence*):** Para rastreamento de preços, pesquisa de mercado e análise de sentimento Khder [2021].
- **Ciência de Dados e Aprendizado de Máquina (*Machine Learning*):** Como método para obter grandes volumes de dados necessários para treinar e avaliar modelos, inclusive em casos onde as APIs são limitadas ou inexistentes Khder [2021]; Reddi et al. [2021].

Essa técnica é particularmente útil quando os dados não são acessíveis por meio de uma Interface de Programação de Aplicações (**API**) ou quando o acesso via API é limitado, pago ou restrito Khder [2021].

4.2 Ingestão dos dados

A escolha da ferramenta de armazenamento para a fase de ingestão é um ponto crítico do *pipeline*. Embora bibliotecas como o **Pandas** e frameworks de processamento distribuído como o **Apache Spark** sejam populares para a engenharia de dados, o **DuckDB** foi a solução preferencial para este projeto, especialmente no que tange à volumetria de dados e ao custo operacional.

4.2.1 Limitações do Pandas e do Apache Spark

O **Pandas** é uma biblioteca de manipulação de dados em memória, o que significa que ele carrega todo o conjunto de dados na RAM para processamento. Essa característica, embora eficiente para bases pequenas e médias, torna-se uma limitação importante em cenários de web scraping em larga escala, pois o consumo de memória cresce proporcionalmente ao volume de dados. Dessa forma, rapidamente os recursos computacionais da máquina podem ser esgotados, tornando o *pipeline* ineficiente ou até mesmo inviável para grandes volumes de informação.

Por outro lado, o **Apache Spark** foi projetado para o processamento de *Big Data* em clusters distribuídos, sendo uma ferramenta extremamente poderosa para lidar com datasets que chegam à ordem de petabytes. No entanto, a configuração e manutenção de um ambiente Spark exigem uma infraestrutura complexa, composta por múltiplos servidores, além de uma curva de aprendizado acentuada. Considerando o escopo deste projeto, que visa demonstrar a viabilidade de *pipelines* acessíveis a pequenas e médias empresas, bem como a projetos acadêmicos, a adoção do Spark não se justificaria devido à sua complexidade e aos custos associados à infraestrutura necessária.

Nesse contexto, a escolha do **DuckDB** foi estratégica. Posicionando-se como um banco de dados analítico embutido, o DuckDB combina a eficiência do processamento em colunas com a simplicidade de uma biblioteca integrada. Sua adoção se deu por três motivos principais. Primeiro, pela otimização para volumetria de dados: ao contrário do Pandas, o DuckDB não precisa carregar todos os dados na memória para processá-los, podendo trabalhar diretamente a partir do disco. Isso permite lidar com arquivos maiores do que a RAM disponível, oferecendo desempenho superior em cenários de médio e grande porte. Após a execução inicial de todos os dez scripts de web scraping desenvolvidos neste trabalho, o volume total de dados brutos coletados e armazenados no banco `destinosbrasil.duckdb` alcançou 3.084 KB. Esse resultado prático valida a discussão teórica e comprova a viabilidade do uso de soluções *open source* em cenários de dados de médio porte.

O segundo motivo refere-se à simplicidade e custo zero. Por ser uma biblioteca de código aberto, o DuckDB pode ser facilmente instalado e integrado a scripts Python, sem a necessidade de um servidor dedicado, eliminando custos de licenciamento e infraestrutura. Por fim, destaca-se a flexibilidade e confiabilidade da ferramenta. O DuckDB se mostra ideal para atuar como um repositório temporário, persistindo os dados de forma estruturada entre a fase de ingestão e

as etapas subsequentes de transformação. Dessa forma, oferece a confiabilidade de um banco de dados relacional aliada à flexibilidade de lidar com dados semi-estruturados, garantindo que o *pipeline* desenvolvido seja robusto, seguro e eficiente.

Com essa abordagem, o *pipeline* de dados demonstra que é possível construir uma arquitetura eficiente e economicamente viável, usando ferramentas *open source* que se adaptam perfeitamente à volumetria e aos requisitos do projeto, sem o peso de infraestruturas complexas ou de alto custo.

4.2.2 Modelagem das tabelas

A modelagem de dados de cada tabela foi desenvolvida de forma personalizada, respeitando a estrutura HTML e a organização de conteúdo específica de cada site. Todos os scripts implementam verificação de duplicidade antes de inserir novos registros, comparando os dados extraídos com registros já existentes no banco. Essa abordagem adaptativa foi necessária devido às diferenças significativas entre as plataformas.

Variações Estruturais e Padronização dos Dados

Problema	Solução
Nível de Detalhamento: Alguns sites (ex.: Journey Latin America, Newmarket Holidays) oferecem informações mais granulares, enquanto outros (ex.: Sol Férias, Panam) apresentam estruturas simples.	Padronização dos campos: criação de estrutura flexível com campos opcionais para dados adicionais e campos obrigatórios para informações essenciais.
Formato de Preços: Alguns sites exibem preços padronizados; outros misturam moeda e texto descritivo.	Utilização de campos de texto para armazenar valores mistos e posterior normalização via scripts de limpeza.
Descrições e Atributos: Sites europeus apresentam descrições estruturadas (highlights, includes), enquanto sites latino-americanos usam formato livre.	Uso combinado de campos de texto curto e longo para suportar diferentes níveis de estruturação.
URLs e Navegação: Variação significativa na estrutura das URLs (deep links, hashes, parâmetros complexos).	Criação de campo URL padronizado e tratamento posterior com scripts de análise para extração de componentes relevantes.
Idioma e Terminologia: Termos específicos em diferentes idiomas (ex.: <i>circuit/séjour</i> , <i>rundreisen</i> , <i>holidays</i>).	Preservação da terminologia original nos campos de tipo/categoria, garantindo consistência sem perda semântica.
Controle Temporal: Necessidade de rastrear quando os dados foram coletados.	Inclusão de um campo de data de extração em todas as tabelas.
Heterogeneidade de Campos: Diferenças entre tabelas quanto à presença de colunas específicas.	Definição mínima comum de colunas (preço, URL, data de extração) para uniformização.
Textos de Diferentes Tamanhos: Campos variando entre descrições curtas e longas.	Utilização de campos de texto com tamanhos adequados a cada tipo de conteúdo.

Tabela 5: Problemas e soluções observados na estrutura dos sites analisados

4.2.3 Implementação Prática

A fase de ingestão de dados, a primeira e crucial etapa do *pipeline*, foi executada com o objetivo de demonstrar a eficácia de soluções *open source* para a coleta de dados brutos. O processo foi inteiramente construído com ferramentas de código aberto, reforçando a premissa central deste trabalho.

4.2.4 Desafios e Estratégias de Mitigação

A fase de ingestão foi a mais complexa do *pipeline*, pois exigiu a superação de diversos desafios técnicos impostos pela natureza variada das fontes de dados. As estratégias de mitigação implementadas demonstraram a resiliência e a adaptabilidade da sua solução:

Durante o desenvolvimento dos scripts de web scraping, diversos desafios técnicos foram enfrentados e solucionados de forma estratégica. Um dos principais foi o conteúdo dinâmico e o carregamento assíncrono. Muitos dos websites analisados utilizavam JavaScript para carregar as ofertas de forma dinâmica, o que impedia a coleta direta dos dados por meio do código-fonte estático da página. Para contornar essa limitação, foi implementada a automação de interações, como o clique em botões "Ver Mais" e a rolagem da página até que todo o conteúdo estivesse visível, garantindo, assim, a extração completa das informações.

Outro obstáculo importante envolveu as defesas anti-automação. Alguns sites aplicavam mecanismos de detecção de robôs, o que exigiu a adoção de medidas como a ocultação de rastros do WebDriver e a implementação de pausas estratégicas entre as ações de navegação e coleta, simulando um comportamento humano e reduzindo a probabilidade de bloqueio.

A variabilidade na estrutura dos websites também representou um desafio significativo, uma vez que cada domínio apresentava diferentes padrões de HTML e seletores CSS. Para lidar com essa heterogeneidade, foi adotada uma abordagem defensiva, na qual os scripts tentavam múltiplos seletores alternativos para cada informação relevante como título, preço e descrição, minimizando a necessidade de refatoração frequente do código.

Nos casos em que os dados estavam distribuídos em múltiplas páginas, foi necessário lidar com navegação e paginação complexas. Para isso, foram implementadas rotinas que incluíam a iteração sobre listas de URLs pré-definidas, a automação de cliques em botões de "próxima página" e, em algumas situações, até a simulação de rolagem horizontal em carrosséis de produtos, garantindo que todos os registros fossem coletados.

Por fim, uma camada adicional de resiliência foi obtida por meio do tratamento de erros e da lógica de *fallback*. Cada script foi projetado com blocos de captura de exceções, assegurando que eventuais falhas não interrompessem o processo completo de coleta. Em caso de erro na extração de determinado elemento, a lógica tentava seletores alternativos e, se ainda assim não obtivesse sucesso, registrava o incidente para posterior diagnóstico. Essa estratégia aumentou significativamente a robustez e a confiabilidade do *pipeline* de scraping desenvolvido.

4.2.5 Formato e Armazenamento dos Dados Brutos

Após a extração, os dados foram estruturados em um formato tabular e armazenados em tabelas dedicadas no **DuckDB**. A criação de uma tabela por fonte (ex: Transalpino, Jetmar, Panam) é uma estratégia de modelagem que separa o dado "cru" do "limpo" e facilita a análise de proveniência, um passo crítico em qualquer projeto de engenharia de dados. Os dados foram persistidos em seu formato original, com a intenção de que a padronização, limpeza e enriquecimento ocorram nas próximas etapas do *pipeline*, o que será documentado nos capítulos seguintes. Dessa forma, a fase de ingestão demonstra não apenas a viabilidade de uma arquitetura baseada em ferramentas *open source*, mas também o seu potencial de aplicação prática em cenários reais, como no suporte à **Embratur** e a outras instituições que necessitam de dados confiáveis para a tomada de decisão estratégica.

4.3 Transformação dos Dados com dbt e PostgreSQL

Após a fase de ingestão, na qual os dados brutos foram armazenados no DuckDB, o *pipeline* avança para a etapa de transformação, seguindo a abordagem ELT discutida no Capítulo 2. Nesta fase, os dados são primeiramente migrados para o PostgreSQL e, em seguida, transformados declarativamente pelo dbt (*data build tool*), que opera exclusivamente dentro do banco de dados destino.

4.3.1 Migração do DuckDB para PostgreSQL

A migração dos dados brutos do DuckDB para o PostgreSQL foi implementada por meio de um script Python dedicado, que utiliza uma funcionalidade nativa do DuckDB: a extensão de conexão com PostgreSQL. Essa extensão permite que o DuckDB se conecte diretamente a um banco PostgreSQL, possibilitando a transferência de dados entre os dois sistemas sem a necessidade de bibliotecas intermediárias. Na prática, o DuckDB “enxerga” as tabelas do PostgreSQL como se fossem suas, permitindo copiar dados de um sistema para o outro com comandos SQL simples.

O processo de migração segue as seguintes etapas: (1) cópia do arquivo do banco DuckDB para um diretório temporário, evitando conflitos caso outro processo esteja acessando o banco ao mesmo tempo; (2) estabelecimento de conexão simultânea com o DuckDB e o PostgreSQL;

(3) para cada uma das dez tabelas de dados brutos, criação da tabela correspondente na área de dados brutos (chamada de *schema raw*) do PostgreSQL, caso ainda não exista; e (4) inserção apenas dos registros novos, ou seja, aqueles cuja data de extração ainda não existe no destino, evitando duplicatas entre execuções consecutivas do *pipeline*.

Essa estratégia de inserção incremental, na qual apenas dados novos são transferidos a cada execução, é particularmente relevante em um *pipeline* orquestrado com execução diária, pois elimina a necessidade de reprocessar todo o histórico a cada dia. A Tabela 6 lista as dez tabelas migradas e seus respectivos nomes no PostgreSQL.

Tabela DuckDB	Tabela PostgreSQL (raw)	País
Bleu_Selectour	raw.bleu_selectour	França
Comptoir_Des_Voyages	raw.comptoir_des_voyages	França
Ikarus_Tours	raw.ikarus_tours	Alemanha
Jetmar	raw.jetmar	Uruguai
Journey_Latin_America	raw.journey_latin_america	Reino Unido
Newmarket_Holidays	raw.newmarket_holidays	Reino Unido
Panam	raw.panam	Chile
Sol_Ferias	raw.sol_ferias	Portugal
Transalpino	raw.transalpino	Portugal
Turismo_Costanera	raw.turismo_costanera	Chile

Tabela 6: Mapeamento das tabelas do DuckDB para o *schema raw* do PostgreSQL

4.3.2 Arquitetura Medallion no PostgreSQL

A organização dos dados no PostgreSQL segue a arquitetura *Medallion* (Bronze/Silver/Gold), um padrão amplamente adotado na engenharia de dados que organiza as informações em camadas progressivas de qualidade e refinamento. Essa arquitetura foi implementada por meio de três áreas de armazenamento separadas (chamadas *schemas*), criadas automaticamente pelo script de inicialização do banco:

- **raw (Bronze):** Primeira camada, contendo os dados brutos exportados do DuckDB exatamente como foram coletados, sem nenhuma transformação. Preserva o dado original para fins de rastreabilidade e, caso necessário, reprocessamento.
- **staging (Silver):** Segunda camada, onde o dbt cria visualizações dos dados já limpos e padronizados. Nesta camada, são aplicadas operações como remoção de caracteres indesejados, conversão de textos de preço para valores numéricos e extração de informações estruturadas a partir de campos de texto livre.

- **marts (Gold):** Terceira e última camada, onde o dbt cria tabelas consolidadas e otimizadas para consumo analítico pelo Apache Superset. Contém a tabela de fatos (com os dados dos pacotes turísticos), as tabelas de dimensões (com informações sobre operadoras, destinos e datas) e tabelas analíticas adicionais para análises exploratórias.

Essa separação em camadas foi configurada no arquivo de configuração do projeto dbt, que define que os modelos da camada staging são criados como *views* (consultas salvas que são recalculadas sob demanda, economizando espaço em disco), enquanto os modelos da camada marts são criados como *tables* (tabelas físicas com dados pré-calculados, otimizando o desempenho das consultas). Uma configuração adicional garante que o dbt utilize os nomes das camadas exatamente como definidos, sem adicionar prefixos automáticos — um comportamento padrão da ferramenta que foi ajustado para manter a nomenclatura organizada.

4.3.3 Camada Staging: Limpeza e Padronização

Foram desenvolvidos dez modelos de staging, um para cada operadora, seguindo um padrão de nomenclatura que identifica claramente a fonte dos dados (por exemplo, `stg_bleu_selectour` para os dados da operadora francesa Bleu Selectour). Cada modelo aplica um conjunto de transformações sobre os dados brutos, padronizando as colunas para um formato comum que viabiliza a unificação na camada seguinte.

As transformações realizadas na camada staging incluem:

1. **Enriquecimento com metadados:** Adição de informações sobre a origem de cada registro, nome da operadora, país de origem e idioma, como valores fixos em cada modelo. Isso permite que, após a unificação de todos os dados em uma única tabela, seja possível identificar de qual site cada pacote turístico foi coletado.
2. **Extração numérica de preços:** Os campos de preço nos dados brutos contêm textos em formatos variados, como “1 230 €”, “£3975pp” ou “desde \$599”. Para viabilizar comparações e cálculos, cada modelo aplica uma expressão regular para remover todos os caracteres que não são dígitos, convertendo o resultado em um valor numérico. Operadoras sul-americanas (Panam, Turismo Costanera e Jetmar) utilizam o padrão de separadores numéricos local, onde o ponto indica milhar e a vírgula indica decimal, exigindo uma etapa adicional de conversão antes da transformação final.

A moeda é identificada separadamente com base no país de origem da operadora: Euro para as operadoras francesas, portuguesas e alemã; libra esterlina para as britânicas. Para a operadora uruguaia (Jetmar) e a operadora chilena Panam, o dólar americano é atribuído diretamente como valor fixo. A operadora chilena Turismo Costanera adota uma abordagem distinta: a moeda é extraída dinamicamente do próprio texto do preço por meio de expressão regular, com o dólar americano como valor padrão quando nenhum símbolo monetário é identificado.

3. **Extração de duração em dias:** O campo de duração dos pacotes contém textos heterogêneos como “15 days from £3975pp” ou “8 jours / 7 nuits” (em francês, “8 dias / 7 noites”). Uma função reutilizável, desenvolvida especificamente para este projeto, extrai o primeiro número encontrado no texto e o interpreta como a duração em dias. Quando nenhum número é encontrado, o campo recebe um valor vazio, evitando erros de processamento. Uma exceção é o modelo `stg_transalpino`, que utiliza uma expressão regular própria para identificar padrões do tipo “*N Dias*” diretamente no campo de descrição, em vez de aplicar a macro compartilhada. Adicionalmente, o modelo `stg_jetmar` atribui valores nulos aos campos de duração, pois o site de origem não disponibiliza essa informação de forma estruturada.
4. **Filtragem de registros inválidos:** Cada modelo remove registros que não possuem título, que contêm marcadores de erro (como “não encontrado” em português ou “nicht angegeben” em alemão), e, no caso de operadoras que oferecem destinos além do Brasil, aplica filtros por palavras-chave no título, na descrição ou na URL para manter apenas os pacotes com destino ao Brasil.

A Tabela 7 apresenta os filtros específicos aplicados nos modelos de operadoras que oferecem destinos múltiplos.

Modelo	Filtro de Destino Brasil
stg_ikarus_tours	Busca pela palavra “Brasil” (em português/espanhol) e também por “Brasilien” (que é Brasil em alemão). Isso garante que, se o anúncio for para turistas europeus, ele ainda seja capturado. Mesmo que o texto não diga o nome do país, o filtro entende que se o roteiro menciona “Amazon” (Amazônia) ou “Iguazu” (Cataratas do Iguazu), a viagem é, com certeza, para o Brasil.
stg_journey_latin_america	O sistema olha para o endereço da página. Se na URL aparecer algo como /brazil/, ele já classifica automaticamente como destino Brasil. Ele também faz uma varredura no texto descritivo buscando pela grafia em inglês: “Brazil”.
stg_transalpino	Filtro por destino Brasil aplicado via CTE intermediária: mantém apenas registros onde o campo de destino (destino) ou a URL do pacote (url) contém a palavra “brasil” (sem distinção de maiúsculas/minúsculas).
stg_turismo_costanera	Remoção de registros com título vazio ou em branco. Filtro estrito pelo campo de localização: apenas registros onde local = 'Brasil' são mantidos, garantindo exclusividade de destino sem necessidade de análise textual.

Tabela 7: Filtros de destino aplicados nos modelos staging

4.3.4 Macros Reutilizáveis

Para evitar a duplicação de lógica entre os dez modelos de staging, foram desenvolvidas duas funções reutilizáveis (chamadas *macros* na terminologia do dbt), armazenadas em um diretório dedicado do projeto:

- **Extração de duração em dias:** Recebe o nome de uma coluna de texto e retorna um número inteiro correspondente ao primeiro número encontrado naquele texto. Por exemplo, a partir do texto “15 days from £3975pp”, a macro extrai o número 15. Quando nenhum dígito é encontrado no texto, retorna um valor vazio, evitando erros de conversão.
- **Deteção automática de moeda:** Recebe o nome de uma coluna de texto e identifica a moeda com base nos símbolos presentes — o símbolo £ indica Libra Esterlina, o símbolo € indica Euro, e a sigla USD indica Dólar Americano. Nos modelos atuais, a moeda é

atribuída diretamente com base no país da operadora ou extraída via expressão regular diretamente nos modelos de staging; esta função permanece disponível como utilitário para cenários futuros onde a detecção automática seja necessária.

4.3.5 Camada Marts: Consolidação e Modelagem Dimensional

A camada marts contém os modelos finais otimizados para consumo analítico, organizados seguindo uma modelagem dimensional — um padrão de organização de dados amplamente utilizado em *Business Intelligence*, no qual os dados são divididos entre tabelas de fatos (que contêm as métricas e medições) e tabelas de dimensões (que contêm os atributos descritivos que contextualizam os fatos). Além dessas, foram criadas tabelas analíticas adicionais para facilitar análises exploratórias e comparativas. Todos os modelos desta camada são armazenados como tabelas físicas no PostgreSQL, garantindo desempenho nas consultas realizadas pelo Superset.

One Big Table (OBT)

O modelo central desta camada é a chamada *One Big Table* (OBT), literalmente “Uma Grande Tabela”. Seu papel é unificar os dados dos dez modelos de staging em uma única tabela consolidada, com colunas padronizadas que incluem: data de extração, nome da operadora, país de origem, idioma, título do pacote, descrição, tipo, texto original do preço, valor numérico do preço, moeda, texto original da duração, duração em dias e URL.

Para identificar cada registro de forma única, é gerada uma chave artificial utilizando uma função de resumo criptográfico (MD5) sobre a combinação do nome da operadora, o título do pacote e a data de extração. Essa abordagem garante que cada registro tenha um identificador exclusivo, sem depender de campos dos sites de origem que podem ser inconsistentes ou inexistentes.

Tabela Fato: fct_pacotes

A tabela de fatos (fct_pacotes) é derivada da OBT e associa cada pacote turístico à sua respectiva operadora por meio de uma chave de referência. A granularidade — ou seja, o nível de detalhe — é de um registro por pacote por data de extração. Cada registro contém os valores de preço e duração como métricas quantitativas, além das chaves que permitem cruzar as informações com as tabelas de dimensões.

Dimensão de Operadoras: `dim_operadora`

A tabela de dimensão de operadoras é alimentada por um arquivo de dados auxiliar (chamado *seed* na terminologia do dbt): uma planilha em formato CSV, elaborada manualmente, contendo os metadados das dez operadoras turísticas — nome, país de origem, idioma e moeda padrão. O modelo enriquece essas informações com campos derivados que facilitam a análise: por exemplo, o código “fr” é traduzido para “Francês”, o código “EUR” é traduzido para “Euro”, e o país “França” é classificado na região “Europa”. Esses campos facilitam a criação de filtros e agrupamentos nos painéis do Superset.

Dimensão Temporal: `dim_tempo`

A tabela de dimensão temporal é gerada automaticamente a partir das datas de extração presentes nos dados, evitando a necessidade de manter manualmente uma tabela de calendário. O modelo decompõe cada data em seus componentes — ano, mês, dia, trimestre, número da semana no ano e nome do dia da semana — e inclui um indicador que identifica se a data corresponde a um fim de semana. Esses atributos permitem análises temporais detalhadas no Superset, como a comparação de preços entre diferentes períodos ou a identificação de padrões sazonais.

Dimensão de Destinos: `dim_destino`

A tabela de dimensão de destinos representa uma das transformações mais complexas do projeto. Como os dados brutos não possuem um campo estruturado de destino — a informação está dispersa nos títulos dos pacotes, escritos em múltiplos idiomas — este modelo implementa uma estratégia de correspondência por palavras-chave em cinco idiomas.

O modelo parte dos dados da OBT aplicando um filtro de moeda: apenas registros com preço em USD, EUR ou GBP são considerados, excluindo os valores em peso chileno (CLP), cuja escala de grandeza distinta inviabilizaria comparações diretas entre moedas. Em seguida, analisa o título de cada pacote e, por meio de uma extensa regra condicional com mais de 80 variações de palavras-chave, classifica o pacote em um dos 21 destinos brasileiros identificados. Por exemplo: os termos “iguaçu” (português), “iguazu” (espanhol), “iguassu” (variação internacional) e “cataratas” (genérico) são todos mapeados para o destino “Foz do Iguaçu”. Os destinos mapeados são: Rio de Janeiro, Bahia/Salvador, Amazônia, Foz do Iguaçu, Pantanal, Fernando de Noronha, Florianópolis/SC, Fortaleza/CE, Natal/Pipa, Maceió/Maragogi,

Porto Seguro/Trancoso, Recife/Nordeste, João Pessoa/RN, Búzios, Angra/Ilha Grande, Lençóis Maranhenses, Porto de Galinhas, São Paulo, Paraty, Gramado/Serra Gaúcha e Brasil (múltiplos destinos). Pacotes que não correspondem a nenhum padrão reconhecido são classificados como “Outros / Múltiplos”.

Essa abordagem, embora funcional e eficaz para o escopo do projeto, evidencia uma das limitações do processamento baseado em regras fixas: a necessidade de manutenção manual das palavras-chave à medida que novos destinos ou variações linguísticas surgem nos sites monitorados.

Mart de Faixas de Preço por Destino: `mart_faixa_precos_destino`

Derivado diretamente da `dim_destino`, este modelo consolida as faixas de preço para cada destino, excluindo as categorias genéricas (“Outros / Múltiplos” e “Brasil (múltiplos destinos)”). Os dados são agrupados por destino e moeda, com o cálculo do preço médio, mínimo e máximo, além do total de pacotes por combinação. Os resultados são ordenados por preço médio decrescente, facilitando a identificação dos destinos mais e menos acessíveis dentro de cada moeda.

Mart de Preços por Operadora: `mart_precos_por_operadora`

Este modelo cruza os dados da tabela de fatos (`fct_pacotes`) com os metadados da dimensão de operadoras (`dim_operadora`), produzindo uma visão analítica comparativa por região de origem, operadora e moeda. Além do filtro padrão de preço nulo ou zero, aplica-se um tratamento de outliers: registros com preço em CLP inferior a 10.000 são descartados, pois representam valores equivalentes a menos de dez dólares americanos, caracterizando dados claramente espúrios. O modelo agrega preço médio, menor preço, preço máximo e total de pacotes para cada combinação de região, operadora e moeda.

4.3.6 Testes de Qualidade de Dados

O projeto dbt implementa testes automáticos de qualidade de dados, definidos em arquivos de configuração das camadas *staging* e *marts*. Esses testes são executados automaticamente ao final de cada execução do *pipeline*, funcionando como uma rede de segurança que verifica se os dados transformados atendem aos critérios de qualidade esperados. Os testes implementados

incluem:

- **Testes de campos obrigatórios:** Verificam que campos essenciais, como data de extração, nome da operadora, título do pacote e identificadores, nunca estejam vazios em nenhum registro.
- **Testes de unicidade:** Garantem que as chaves primárias das tabelas de dimensões (como o identificador de cada operadora e de cada data) não contenham valores duplicados, preservando a integridade do modelo dimensional.
- **Testes de valores permitidos:** Verificam que o campo de moeda contenha apenas os códigos esperados, EUR (Euro), GBP (Libra Esterlina), USD (Dólar Americano) e CLP (Peso Chileno), tanto nos modelos de staging individuais quanto na tabela unificada.

No total, o projeto conta com mais de 40 testes distribuídos entre as camadas staging e marts, cobrindo a integridade referencial, a consistência dos campos monetários e a completude dos registros. Caso algum teste falhe, a execução do *pipeline* é interrompida e o problema é registrado nos logs do Airflow para investigação.

4.3.7 Arquitetura da DAG

O *pipeline* completo foi modelado como uma única DAG (*Directed Acyclic Graph* — Grafo Direcionado Acíclico) no Airflow, denominada `pipeline_destinos_brasil`. Uma DAG é, em termos simples, um mapa que define quais tarefas devem ser executadas e em qual ordem, garantindo que nenhuma tarefa comece antes que suas dependências tenham sido concluídas. A DAG deste projeto é composta por seis etapas sequenciais:

1. **Scrapers (sequenciais):** Dez tarefas que executam os scripts de *web scraping*, uma para cada operadora. As tarefas são encadeadas sequencialmente, ou seja, uma só começa após a anterior terminar, para evitar a sobrecarga de memória e de conexões simultâneas, já que cada tarefa inicia um navegador Chrome completo dentro de um container.
2. **Exportação DuckDB para PostgreSQL:** Uma tarefa que executa o script de migração, transferindo os dados brutos para a camada raw do PostgreSQL.

3. **Carga dos dados auxiliares (dbt seed):** Carrega o arquivo com os metadados das operadoras como tabela no PostgreSQL, fornecendo as informações de referência necessárias para a dimensão de operadoras.
4. **Transformação da camada staging (dbt run staging):** Cria as dez visualizações de dados limpos e padronizados na camada staging.
5. **Transformação da camada marts (dbt run marts):** Cria as tabelas consolidadas, a tabela unificada (OBT), a tabela de fatos e as tabelas de dimensões, na camada marts.
6. **Testes de qualidade (dbt test):** Executa todos os testes automáticos para validar a integridade dos dados transformados.

4.3.8 Uso do DockerOperator

Uma decisão arquitetural importante foi a escolha de executar cada tarefa do Airflow dentro de um container Docker independente e temporário, em vez de executá-las diretamente no ambiente do Airflow. Na prática, isso significa que, a cada tarefa, o Airflow cria um ambiente isolado, executa o trabalho necessário e, ao final, remove esse ambiente automaticamente, liberando os recursos da máquina.

Essa abordagem, viabilizada pelo componente DockerOperator do Airflow, oferece três vantagens significativas para este projeto:

- **Isolamento de dependências:** O ambiente dos scrapers inclui o navegador Google Chrome e a biblioteca Selenium, enquanto o ambiente do dbt inclui apenas as ferramentas de transformação de dados. Sem esse isolamento, todas essas dependências precisariam coexistir no mesmo ambiente, aumentando a complexidade e o risco de conflitos entre versões incompatíveis de bibliotecas.
- **Reprodutibilidade:** O mesmo ambiente utilizado durante o desenvolvimento local é exatamente o mesmo executado pelo Airflow em produção, eliminando o problema clássico de inconsistências entre ambientes.
- **Gerenciamento de recursos:** Cada tarefa de scraping roda em um ambiente isolado com seu próprio processo de navegador. Ao finalizar, o ambiente é removido automaticamente, liberando memória e capacidade de processamento para as tarefas seguintes.

Para viabilizar essa abordagem, o Airflow acessa o sistema de gerenciamento de containers do computador hospedeiro, e os dados são compartilhados entre os diferentes ambientes por meio de diretórios montados: os scripts e o banco DuckDB são acessíveis tanto pelos containers de scraping quanto pelo container de exportação, enquanto os arquivos do projeto dbt são acessíveis pelo container de transformação.

4.3.9 Configuração do Agendamento

A DAG foi configurada para execução diária às 06h no fuso horário de Brasília. Essa periodicidade reflete a frequência desejada de atualização dos dados de passagens aéreas. Uma configuração adicional impede que o Airflow tente executar retroativamente os dias anteriores à criação da DAG, comportamento que seria desnecessário neste projeto. Cada tarefa possui configuração de reexecução automática: em caso de falha, o Airflow aguarda dez minutos e tenta novamente, com um limite máximo de duas horas por tarefa como proteção contra execuções que excedam o tempo esperado, um cenário possível nas tarefas de *web scraping*, que dependem da disponibilidade e da velocidade de resposta dos sites externos.

4.3.10 Configuração do Executor

O Airflow foi configurado com o modo de execução local (LocalExecutor), que permite a execução de tarefas em processos separados na mesma máquina. Essa configuração oferece um equilíbrio entre simplicidade e desempenho: é mais eficiente do que o modo sequencial (que executa apenas uma tarefa por vez), mas não requer a complexidade de um sistema distribuído com múltiplos servidores. O banco de dados de controle do Airflow, onde ele armazena o histórico de execuções, os status das tarefas e as configurações, é hospedado no mesmo PostgreSQL do projeto, porém em um banco de dados separado, evitando qualquer interferência com os dados analíticos.

4.3.11 Resultados da Orquestração

A execução completa da DAG pipeline_destinos_brasil foi realizada com sucesso em abril de 2026, com duração total de 19 minutos e 20 segundos. Todas as 16 tarefas foram concluídas sem falhas, validando a robustez da arquitetura de orquestração proposta. A Figura 3 apresenta

a estrutura visual da DAG na interface do Airflow, exibindo o encadeamento sequencial das tarefas, enquanto a Figura 4 apresenta os detalhes da execução.

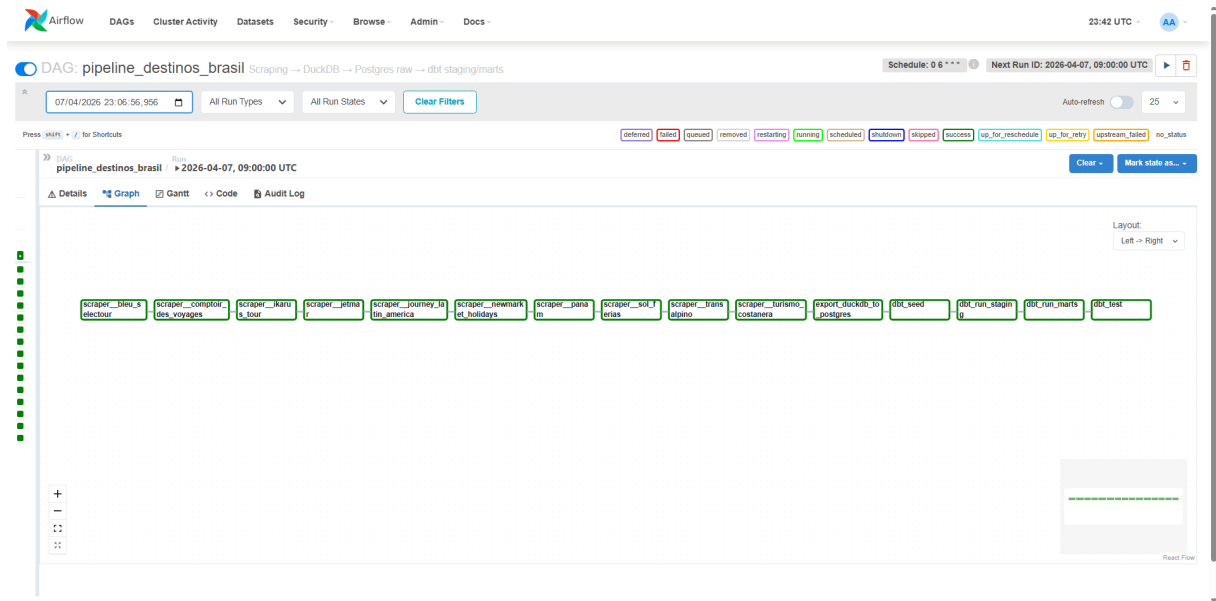


Figura 3: Estrutura visual da DAG pipeline_destinos_brasil na interface do Apache Airflow, exibindo o encadeamento sequencial das 16 tarefas.

Dag Run Details	
Status	success
Run ID	manual__2026-04-07T23:06:56:956029+00:00
Run type	manual
Run duration	00:19:20
Last scheduling decision	2026-04-07, 23:26:18 UTC
Queued at	2026-04-07, 23:06:57 UTC
Started	2026-04-07, 23:06:57 UTC
Ended	2026-04-07, 23:26:18 UTC
Data interval start	2026-04-06, 09:00:00 UTC
Data interval end	2026-04-07, 09:00:00 UTC
Externally triggered	True
Run config	None

Figura 4: Detalhes da execução da DAG, com status de sucesso e duração total de 19 minutos e 20 segundos.

A análise dos tempos de execução revela que a etapa de *web scraping* consome a maior parte do tempo total do pipeline, correspondendo a aproximadamente 92% da duração total (17 minutos e 53 segundos de um total de 19 minutos e 20 segundos). Esse resultado é esperado,

uma vez que os scrapers dependem da velocidade de resposta dos sites externos, do carregamento de conteúdo dinâmico via JavaScript e da simulação de interações humanas com pausas estratégicas para evitar bloqueios. A Tabela 8 detalha o tempo de execução de cada tarefa, organizado por etapa do pipeline.

Etapa	Tarefa	Duração
Scraping	Bleu Selectour	0min 38s
	Comptoir des Voyages	0min 18s
	Ikarus Tours	1min 05s
	Jetmar	7min 25s
	Journey Latin America	1min 22s
	Newmarket Holidays	0min 56s
	Panam	0min 45s
	Sol Férias	2min 31s
	Transalpino	1min 48s
	Turismo Costanera	1min 05s
Subtotal Scraping		17min 53s
Migração	Export de DuckDB para PostgreSQL	0min 05s
Transformação	dbt seed (dados auxiliares)	0min 13s
	dbt run staging (10 modelos)	0min 14s
	dbt run marts (5 modelos)	0min 11s
Qualidade	dbt test (mais de 40 testes)	0min 13s
Total do Pipeline		19min 20s

Tabela 8: Tempo de execução de cada tarefa da DAG pipeline_destinos_brasil

Dentro da etapa de scraping, a tarefa do Jetmar se destaca como a mais demorada (7 minutos e 25 segundos), representando sozinha cerca de 38% do tempo total do pipeline. Essa duração elevada pode ser atribuída à complexidade da estrutura do site, que exige múltiplas interações para revelar todo o conteúdo disponível. Em contrapartida, o Comptoir des Voyages foi o scraper mais rápido, concluindo em apenas 18 segundos.

As etapas de migração e transformação, por outro lado, são notavelmente rápidas. A exportação dos dados do DuckDB para o PostgreSQL levou apenas 5 segundos, demonstrando a eficiência da extensão nativa de conexão entre os dois bancos. As três etapas do dbt, carga dos dados auxiliares, transformação da camada staging e transformação da camada marts, totalizaram 38 segundos, evidenciando que o dbt é capaz de processar os dez modelos de staging, a tabela unificada, a tabela de fatos e as três tabelas de dimensões em tempo desprezível quando comparado à etapa de coleta. Os testes de qualidade, que validaram mais de 40 critérios de integridade dos dados, foram executados em 13 segundos.

Esses resultados demonstram que a arquitetura proposta é viável para execução diária, o

pipeline completo conclui em menos de 20 minutos. A concentração do tempo na etapa de *scraping* também indica que eventuais otimizações futuras, como a paralelização dos *scrapers* ou a substituição de sites com tempos de resposta elevados, teriam o maior impacto na redução do tempo total de execução.

4.4 Visualização dos Dados com Apache Superset

4.4.1 Escolha e Configuração da ferramenta

A última etapa do *pipeline* é a disponibilização dos dados transformados para análise visual interativa. O Apache Superset, versão 3.1.0, foi selecionado como ferramenta de *Business Intelligence* por ser uma plataforma *open source* (licença Apache 2.0) que permite a criação de painéis interativos e a exploração de dados via consultas SQL, posicionando-se como alternativa direta a ferramentas proprietárias como Tableau e Power BI.

O Superset foi configurado por meio do Docker Compose com dois serviços, o serviço principal da aplicação (acessível pela porta 8088 do navegador) e um serviço temporário de inicialização, responsável por preparar o banco de dados interno do Superset e criar o usuário administrador. O banco de dados interno do Superset, onde ele armazena as definições de painéis, gráficos e conexões, utiliza um banco PostgreSQL próprio, separado tanto do banco de dados analítico quanto do banco do Airflow. A interface foi configurada para exibição em português brasileiro.

A conexão com os dados analíticos é realizada diretamente ao banco destinosbrasil, apontando para a camada marts que contém as tabelas consolidadas pelo dbt. Os conjuntos de dados registrados no Superset incluem a tabela unificada (para análises exploratórias diretas, sem necessidade de cruzamento entre tabelas), a tabela de fatos (para análises dimensionais com cruzamento de informações de operadoras, destinos e datas), e as tabelas de dimensões.

4.4.2 Dashboards Desenvolvidos

Os seguintes painéis interativos foram projetados para demonstrar o potencial analítico dos dados coletados e transformados pelo *pipeline*:

Dashboard 1 — Panorama Geral de Passagens para o Brasil: Indicadores-chave como o total de pacotes coletados, o preço médio por moeda e o número de destinos brasileiros iden-

tificados. Gráficos de barras com os destinos mais ofertados e os preços médios por destino. Gráfico de séries temporais mostrando a evolução da coleta ao longo das datas de extração.

Dashboard 2 — Comparação entre Operadoras: Tabela comparativa com preço médio, mínimo e máximo por operadora, agrupados por região de origem (Europa vs. América do Sul). Gráficos de barras agrupadas comparando preços por destino entre diferentes operadoras. Filtros interativos por país de origem, moeda e faixa de preço.

4.4.3 Resultados da Visualização e Análise dos Dados

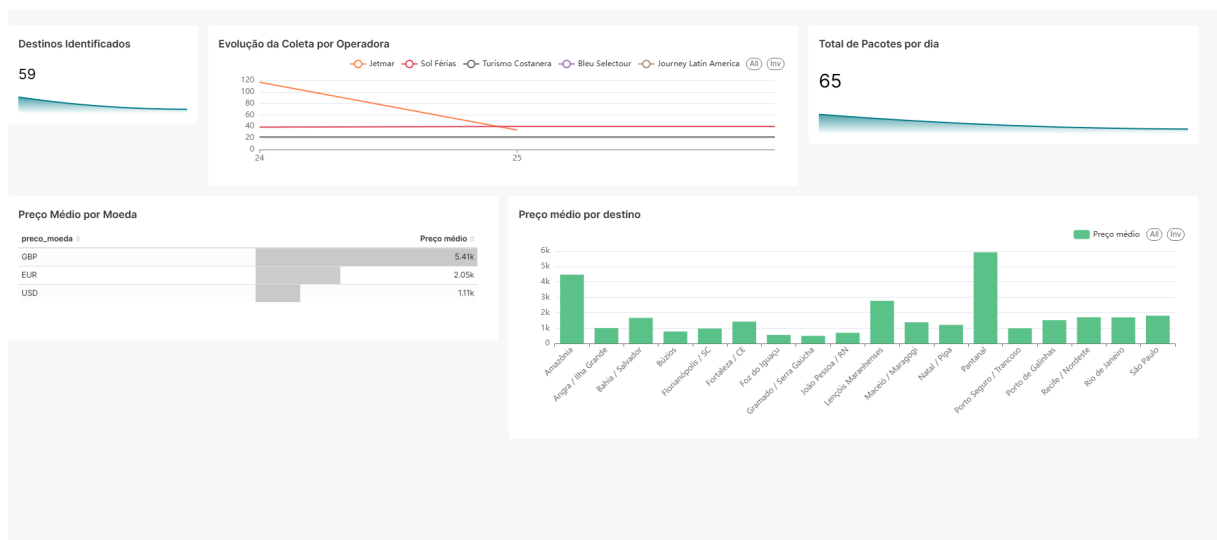


Figura 5: Painei *Panorama Geral* no Apache Superset

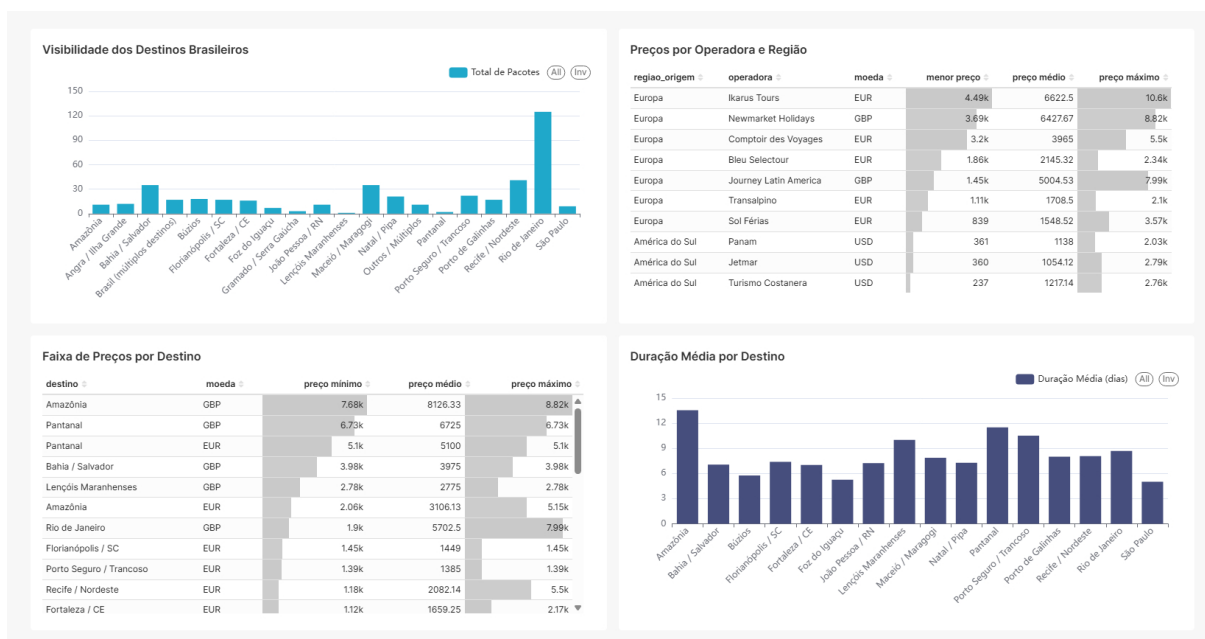


Figura 6: Painel *Detalhamento por Destino* no Apache Superset

A camada de visualização do *pipeline* é composta por dois painéis analíticos implementados no Apache Superset, conectado diretamente ao PostgreSQL. O painel **Panorama Geral** oferece uma visão consolidada da coleta, enquanto o painel **Detalhamento por Destino** aprofunda a análise por destino, operadora e faixa de preços. Juntos, os dois painéis totalizam nove gráficos distribuídos sobre quatro tabelas da camada marts: *fct_pacotes*, *dim_destino*, *mart_faixa_precos_destino* e *mart_precos_por_operadora*.

Painel Panorama Geral

O painel *Panorama Geral* é organizado em duas linhas de visualizações. A primeira linha reúne três indicadores operacionais. O primeiro é o número de destinos identificados, exibido como um grande indicador numérico que computa a contagem distinta de títulos de pacotes ao longo do tempo, acompanhado de uma linha de tendência. O segundo é a evolução do volume de coleta por operadora, representada como um gráfico de linhas temporais que exibe o número de registros coletados por dia para cada uma das dez operadoras. E por último, o total de pacotes coletados por dia, também como um indicador numérico com linha de tendência.

A segunda linha apresenta duas visualizações analíticas: uma tabela com o preço médio agregado por moeda, permitindo comparar diretamente a faixa de preços praticada em Euro, Libra Esterlina e Dólar Americano. Além de um gráfico de barras com o preço médio por destino, derivado da *dim_destino* e com filtro explícito para excluir a categoria “Outros / Múlti-

plos”, ordenado de forma decrescente para evidenciar os destinos com maior valor médio de oferta internacional.

A evolução da coleta por operadora é particularmente útil para fins de monitoramento operacional: quedas bruscas no volume de uma operadora em determinada data sinalizam falhas de raspagem que exigem investigação, enquanto a estabilidade consistente de todas as séries indica que o *pipeline* está operando normalmente. A comparação das linhas entre operadoras também revela diferenças estruturais no catálogo de cada site: operadoras com portfólio menor produzem séries com amplitude reduzida, enquanto as de maior oferta apresentam volumes consistentemente mais elevados.

A tabela de preço médio por moeda permite uma observação imediata da diferença de escala entre os mercados monitorados. As operadoras europeias, que cobram em Euro ou Libra Esterlina, tendem a apresentar valores nominais mais altos do que as sul-americanas, que cobram em dólar americano. Essa diferença, no entanto, não implica necessariamente que as viagens sejam mais caras em termos de poder de compra, uma vez que os pacotes europeus geralmente incluem trechos de voo intercontinental, enquanto os sul-americanos embarcam de países com menor distância ao Brasil.

Os cinco gráficos deste painel compartilham filtros cruzados globais. Clicar em uma barra do gráfico de destinos, por exemplo, propaga automaticamente um filtro temporal ou de forma categórica para os demais gráficos da tela, permitindo investigações interativas sem necessidade de configurar filtros de maneira manual.

Painel Detalhamento por Destino

O painel *Detalhamento por Destino* organiza quatro gráficos em duas linhas, com dois filtros nativos configurados na lateral: um filtro de moeda e um filtro de destino, ambos com seleção múltipla, que propagam seus valores para todos os gráficos do painel simultaneamente.

A primeira linha exibe o gráfico **Visibilidade dos Destinos Brasileiros**, um gráfico de barras que soma o campo `total_pacotes` por destino, ordenado de forma decrescente, revelando quais destinos recebem maior atenção das operadoras internacionais. Já a tabela **Preços por Operadora e Região**, que consolida menor preço, preço médio e preço máximo por combinação de região de origem, operadora e moeda, a partir do `mart_precos_por_operadora`.

A segunda linha apresenta a tabela **Faixa de Preços por Destino**, derivada do `mart_faixa_precos_destino` com colunas de preço mínimo, médio e máximo agrupadas por destino e moeda. O gráfico de

barras **Duração Média por Destino**, que exibe a média do campo de duração de dias para cada destino, excluindo registros sem duração informada.

O gráfico de **visibilidade dos destinos** é o principal instrumento analítico para avaliar a promoção diferencial do Brasil no mercado internacional. A distribuição esperada segue um padrão de cauda longa, no qual um pequeno conjunto de destinos consolidados, principalmente Rio de Janeiro, Amazônia e Foz do Iguaçu, concentra a maior parte dos pacotes oferecidos, ao passo que destinos menos conhecidos internacionalmente, como Lençóis Maranhenses, Gramado ou João Pessoa, aparecem com representação marginal. Essa diferença quantifica, de forma objetiva, a diferença entre a diversidade real de destinos turísticos brasileiros e aquela que é efetivamente explorada pelo mercado internacional atualmente.

A leitura combinada do gráfico de visibilidade com a tabela de faixas de preço permite identificar padrões relevantes, sobre como destinos com alta visibilidade e alta amplitude de preço, como Rio de Janeiro, indicam mercados maduros com oferta diversificada atendendo a múltiplos segmentos de renda. Destinos com baixa visibilidade mas preço médio elevado, como Fernando de Noronha, sugerem nichos de mercado de luxo com oferta restrita. Os destinos com baixa visibilidade e preço médio acessível podem representar oportunidades de promoção internacional ainda pouco exploradas.

O gráfico de **duração média por destino** acrescenta uma dimensão qualitativa relevante: destinos que exigem deslocamentos internos significativos, como a Amazônia ou o Pantanal, tendem a apresentar durações médias mais longas, refletindo a necessidade de roteiros de imersão. Destinos urbanos como Rio de Janeiro e Salvador, por sua vez, apresentam maior variabilidade, pois são frequentemente combinados com outros pontos em pacotes de múltiplos destinos.

Usabilidade e Discussão Crítica

O Apache Superset demonstrou-se adequado para o perfil de uso previsto pelo projeto, que combina a necessidade de monitoramento operacional do *pipeline* com a exploração analítica dos dados de turismo. A configuração em português do Brasil e o uso de sessões persistentes de 30 dias contribuem para uma experiência de uso mais fluída, especialmente relevante para usuários que retornam regularmente aos painéis.

A funcionalidade de filtros cruzados, habilitada globalmente no painel *Panorama Geral* e por filtros nativos no painel *Detalhamento por Destino*, representa um diferencial expressivo em

relação a visualizações estáticas. Ela permite que um analista de turismo, sem conhecimento de SQL, navegue pelos dados de forma exploratória, selecionando uma moeda ou um destino específico e observando instantaneamente como as demais métricas respondem. Essa interatividade é, no contexto do projeto, uma proxy da análise que um especialista em inteligência de mercado realizaria manualmente.

Por outro lado, o Superset apresenta limitações que devem ser consideradas para o público-alvo. A curva de aprendizado para a criação e modificação de gráficos, em contraste com ferramentas comerciais como Tableau ou Power BI, pode representar uma barreira para usuários sem experiência prévia com ferramentas de *Business Intelligence* de código aberto. A ausência de conversão cambial automática entre as três moedas do projeto (USD, EUR e GBP) implica que comparações intermoedas devem ser realizadas com cautela e requerem uma etapa interpretativa manual por parte de quem for fazer a análise. Adicionalmente, por se tratar de uma solução implantada em infraestrutura local via Docker, a disponibilidade do ambiente depende da operação contínua da pilha de contêineres, o que demanda competência técnica para manutenção que pode não estar presente em organizações menores do setor de turismo.

No geral, a combinação de dois painéis com escopo complementar, um operacional e um analítico, atende ao duplo propósito do projeto, que é verificar a integridade da coleta e extrair inteligência de mercado a partir dos dados agregados. A escolha do Apache Superset como ferramenta de visualização se mostrou bastante coerente com os objetivos do projeto, viabilizando análises interativas sem exigir programação por parte do usuário final, ao custo de uma maior complexidade de implantação e manutenção em relação a alternativas comerciais.

4.5 Infraestrutura Containerizada com Docker

Toda a infraestrutura do *pipeline* foi containerizada utilizando o **Docker** e orquestrada via **Docker Compose**. Em termos práticos, a containerização significa que cada componente do sistema (o banco de dados, os scrapers, o dbt, o Airflow e o Superset) roda dentro de um ambiente isolado chamado *container*, que empacota todas as dependências necessárias para o seu funcionamento. O Docker Compose é a aplicação que coordena a criação e a comunicação entre todos esses containers a partir de um único arquivo de configuração, o `docker-compose.yml`.

O arquivo de configuração do projeto define oito serviços organizados em uma rede interna compartilhada, que permite que todos os containers se comuniquem entre si utilizando nomes simples. A Tabela 9 detalha os serviços configurados.

Tabela 9: Serviços definidos no arquivo docker-compose.yml do projeto

Serviço	Imagem / Origem	Função
postgres	postgres:15	Banco de dados principal, hospedando os dados analíticos, os metadados do Airflow e os metadados do Superset
scraper	python:3.11-slim*	Ambiente com navegador Chrome, Selenium e scripts de web scraping
airflow-init	apache/airflow:2.9.0	Serviço temporário que prepara o banco do Airflow e cria o usuário administrador
airflow-webserver	apache/airflow:2.9.0	Interface web do Airflow, acessível pela porta 8080
airflow-scheduler	apache/airflow:2.9.0	Agendador responsável por disparar as execuções das DAGs nos horários configurados
dbt	python:3.11-slim*	Ambiente com a ferramenta dbt para execução das transformações de dados
superset	apache/superset:3.1.0	Interface de Business Intelligence, acessível pela porta 8088
superset-init	apache/superset:3.1.0	Serviço temporário que prepara o banco do Superset e cria o usuário administrador

* Imagens construídas localmente a partir de python:3.11-slim, nomeadas pipeline_scraper e pipeline_dbt, respectivamente.

Dois ambientes customizados foram desenvolvidos especificamente para o projeto. O primeiro constrói o ambiente dos scrapers, partindo de uma imagem base de Python e adicionando o navegador Google Chrome e as bibliotecas necessárias (Selenium, DuckDB e Loguru para registro de logs). O segundo constrói o ambiente do dbt, também a partir da mesma linguagem de programação, instalando exclusivamente o pacote dbt-postgres==1.8.0.

Os dados são persistidos de duas formas complementares. *Volumes* Docker, áreas de armazenamento gerenciadas pelo Docker, são utilizados para o banco de dados PostgreSQL e para os metadados do Superset, garantindo que as informações sobrevivam ao reinício dos containers. *Bind mounts*, mapeamentos diretos entre pastas do computador hospedeiro e pastas dentro dos containers, são utilizados para os diretórios de código e dados, permitindo que alterações feitas nos scripts ou nos modelos dbt sejam imediatamente refletidas dentro dos containers, sem necessidade de reconstrução.

O PostgreSQL desempenha um papel central na arquitetura, hospedando três bancos de dados distintos dentro da mesma instância: o banco analítico (com as camadas *raw*, *staging* e *marts*), o banco de controle do Airflow (com o histórico de execuções e configurações) e o

banco de controle do Superset (com as definições de painéis e conexões). Essa consolidação em uma única instância simplifica a infraestrutura e reduz o consumo de recursos, sendo adequada para o escopo do projeto. Em um cenário de produção com maior escala, cada serviço poderia utilizar uma instância dedicada de banco de dados.

Essa abordagem containerizada garante que o *pipeline* completo possa ser reproduzido em qualquer máquina com Docker instalado por meio do comando *docker compose up*, eliminando conflitos de dependências e reduzindo significativamente a barreira técnica para a validação dos resultados deste trabalho.

Capítulo 5

Conclusão

5.1 Estimativa de Custos Operacionais

A adoção de ferramentas de código aberto elimina os custos de licenciamento de *software*, que em soluções proprietárias equivalentes podem representar a parcela mais expressiva do orçamento de um *pipeline* de dados. No entanto, a ausência de licenças não implica ausência de custos. Os gastos com infraestrutura, servidores, armazenamento e transferência de dados persistem e devem ser estimados com precisão para que o benefício econômico do *open source* seja quantificado de forma correta e justa.

5.1.1 Premissas da Estimativa

A comparação entre a solução de código aberto e os equivalentes gerenciados nas principais nuvens públicas exige um conjunto de premissas explícitas para garantir que o confronto seja reproduzível e justo.

Região geográfica equivalente: As três provedoras avaliadas, Amazon Web Services (AWS), Microsoft Azure e Google Cloud Platform (GCP), oferecem infraestrutura na Região Metropolitana de São Paulo. Para este trabalho, foram selecionadas as seguintes regiões: sa-east-1 (AWS), Brazil South (Azure) e southamerica-east1 (GCP), garantindo paridade de latência e conformidade com requisitos de residência de dados no Brasil Amazon Web Services [2026]; Microsoft Azure [2026]; Google Cloud [2026].

Configuração mínima viável: A execução simultânea dos contêineres Docker que compõem o *pipeline*, com Apache Airflow (agendador, servidor web e executor), PostgreSQL, Apache Superset e os *scrapers* Selenium, requer no mínimo 2 vCPUs e 8 GB de RAM. Essa configuração é adotada como referência para todas as instâncias de máquina virtual comparadas.

Volume de dados: Cada execução completa do *pipeline* coletou 3.084 KB de dados brutos em média. Projetando execuções diárias ao longo de 30 dias, o volume mensal estimado é de

aproximadamente 90 MB, incluindo dados transformados, metadados do Airflow e histórico de execuções no PostgreSQL.

Armazenamento em disco: Será adotado 100 GB de disco SSD para acomodar os dados coletados, o banco PostgreSQL com histórico de 12 meses de execuções, os arquivos de log do Airflow e as imagens dos contêineres Docker.

Janela de operação: As instâncias de *VM* são tratadas como instâncias *on-demand* ativas 24 horas por dia, 7 dias por semana (730 horas por mês), sem planos de reserva (*Reserved Instances*, *Savings Plans* ou *Committed Use Discounts*).

Os valores de referência são aqueles publicados nas páginas oficiais de preços das três provedoras, na modalidade *pay-as-you-go*, vigentes em maio de 2026, ano de publicação deste trabalho Amazon Web Services [2026]; Microsoft Azure [2026]; Google Cloud [2026]. Todos os valores são expressos em dólares norte-americanos (US\$).

5.1.2 Infraestrutura de Servidor

A Tabela 10 compara o custo mensal de uma instância de computação com 2 vCPUs e 8 GB de RAM nas três provedoras, nas regiões sul-americanas de referência.

Instância	vCPU	RAM	Provedor	Região	US\$/hora	US\$/mês
t3.large	2	8 GB	AWS	sa-east-1	0,1344	98,11
D2s v3	2	8 GB	Azure	Brazil South	0,1590	116,07
e2-standard-2	2	8 GB	GCP	southamerica-east1	0,1064	77,67

Tabela 10: Custo mensal de instância de computação 2 vCPU / 8 GB nas regiões sul-americanas, modalidade *on-demand*, Linux (maio/2026).

A instância equivalente na GCP é aproximadamente 21% mais barata do que na AWS e 33% mais barata do que na Azure, para a mesma configuração de hardware, o que reflete as distintas estratégias de precificação regional de cada provedora.

Para a alternativa de serviços gerenciados, o componente de maior impacto financeiro é o orquestrador Apache Airflow gerenciado. A Tabela 11 apresenta as tarifas de referência para esta categoria.

Serviço Gerenciado	Provedor	Região	US\$/mês
Amazon MWAA ^(a) (Small + 1 worker)	AWS	us-east-1 ^(a)	397,85
Apache Airflow via Azure Container Apps (estimado)	Azure	Brazil South	aprox. 100,00
Cloud Composer 2 (ambiente pequeno)	GCP	southamerica-east1	aprox. 200,00

Tabela 11: Custo mensal do Apache Airflow gerenciado nas principais provedoras de nuvem (maio/2026).

^(a) O Amazon MWAA (*Managed Workflows for Apache Airflow*) não está disponível na região sa-east-1 em maio de 2026. o valor de referência é da região us-east-1, a menor tarifa disponível. Inclui um ambiente *small* (US\$ 0,49/hora) e um *worker* adicional (US\$ 0,055/hora), 730 horas/mês Amazon Web Services [2026].

A diferença entre hospedar o Apache Airflow em uma VM própria, incluso no custo da instância, e contratar um serviço gerenciado equivalente é expressiva. Somente no componente de orquestração, a solução gerenciada da AWS representa um gasto superior a US\$ 4.700 por ano, mais de cinco vezes o custo anual da VM inteira na opção mais econômica (GCP: US\$ 77,67 por mês, o que equivale a US\$ 932 por ano).

5.1.3 Armazenamento

A Tabela 12 apresenta as tarifas de armazenamento em disco bloco (SSD) e em objeto para as três regiões de referência.

Tipo	Provedor	Serviço	Região	US\$/GB/mês
Bloco SSD	AWS	EBS gp3	sa-east-1	0,112
Bloco SSD	Azure	Premium SSD v2 LRS	Brazil South	0,152
Bloco SSD	GCP	Persistent Disk SSD	southamerica-east1	0,170
Objeto	AWS	S3 Standard	sa-east-1	0,0405
Objeto	Azure	Blob Storage Hot LRS	Brazil South	0,0326
Objeto	GCP	Cloud Storage Standard	southamerica-east1	0,035

Tabela 12: Tarifas de armazenamento por tipo e provedora nas regiões sul-americanas (maio/2026).

Para os 100 GB de armazenamento em bloco (SSD) adotados como premissa, o custo mensal resulta em US\$ 11,20 (AWS), US\$ 15,20 (Azure) e US\$ 17,00 (GCP). O armazenamento em objeto, que é comumente utilizado nos serviços gerenciados para *data lakes* e ingestão de dados em escala, apresenta tarifas de 2,5 a 5 vezes menores que o armazenamento em bloco. Nessa categoria, o Azure Blob Storage Hot LRS é o mais econômico entre as três provedoras avaliadas.

5.1.4 Transferência de Dados

A Tabela 13 apresenta as tarifas de transferência de dados de saída (*egress*) para a internet nas três regiões de referência.

Provedor	Região	Franquia gratuita	US\$/GB (acima)
AWS	sa-east-1	1 GB/mês	0,250
Azure	Brazil South	100 GB/mês	0,120
GCP	southamerica-east1	—	0,230

Tabela 13: Tarifas de transferência de dados de saída (*egress*) para a internet nas regiões sul-americanas, modalidade *on-demand* (maio/2026).

Para o volume coletado pelo projeto deste trabalho, aproximadamente 90 MB mensais com execuções diárias, o custo de transferência é negligenciável em todas as provedoras. Contudo, em cenários com múltiplos usuários acessando *dashboards* ou com volumes de coleta substancialmente maiores, o *egress* passa a ser fator relevante. A tarifa da AWS (US\$ 0,250/GB) é 2,1 vezes superior à da Azure (US\$ 0,120/GB), diferença que deve ser considerada em arquiteturas com consultas frequentes a dados armazenados em nuvem.

5.1.5 Custo Total e Comparação com Soluções Proprietárias

A Tabela 14 consolida os custos mensais estimados para dois cenários: (a) implantação da solução *open source* em uma VM própria na respectiva nuvem e (b) substituição de cada componente pelo serviço gerenciado equivalente da mesma provedora.

Componente / Cenário	AWS sa-east-1	Azure Brazil South	GCP s.a.-east1	Referência
<i>Cenário A — Open Source auto-hospedado (US\$/mês)</i>				
VM (2 vCPU, 8 GB RAM)	98,11	116,07	77,67	Tab. 10
Armazenamento (100 GB SSD)	11,20	15,20	17,00	Tab. 12
Total — Cenário A	109,31	131,27	94,67	
<i>Cenário B — Serviços gerenciados equivalentes (US\$/mês)</i>				
Orquestração (Airflow)	397,85 ^(a)	aprox. 100,00	aprox. 200,00	Tab. 11
PostgreSQL gerenciado	110,78	104,23	121,68	
Visualização (BI)	18,00 ^(b)	13,70 ^(c)	0,00 ^(d)	
Transformações (dbt)	aprox. 2,20 ^(e)	aprox. 5,00 ^(f)	0,00 ^(g)	
Armazenamento (100 GB)	11,20	15,20	17,00	Tab. 12
Total — Cenário B	aprox. 540	aprox.238	aprox. 339	
Fator B / A	4,9 vezes	1,8 vezes	3,6 vezes	

Tabela 14: Comparativo de custo mensal entre a solução *open source* auto-hospedada (Cenário A) e os serviços gerenciados equivalentes (Cenário B) nas três principais provedoras de nuvem, nas regiões sul-americanas (maio/2026).

^(a) Amazon MWAA Small em us-east-1 (não disponível em sa-east-1): ambiente US\$ 0,49/h + 1 worker US\$ 0,055/h por 730 h/mês.

^(b) Amazon QuickSight com um autor.

^(c) Microsoft Power BI Pro com um usuário.

^(d) Looker Studio no plano gratuito.

^(e) AWS Glue com 30 execuções mensais de 5 minutos (2 DPU's).

^(f) Azure Data Factory, com uso leve de Integration Runtime. ^(g) Dataform incluído sem custo no BigQuery.

Os resultados evidenciam que a solução *open source* auto-hospedada custa entre US\$ 94,67 (GCP) e US\$ 131,27 (Azure) por mês, enquanto os equivalentes gerenciados variam de US\$ 238 (Azure) a US\$ 540 (AWS). O fator de custo, a razão entre o Cenário B e o Cenário A na mesma provedora, oscila de 1,8 vezes (Azure) a 4,9 vezes (AWS), chegando a 5,7 vezes quando se compara a opção *open source* mais econômica (GCP: US\$ 94,67) ao equivalente gerenciado

mais caro (AWS: US\$ 540).

A diferença menos expressiva no caso da Azure (1,8 vezes) decorre de um fator estrutural. A plataforma não oferece um serviço de Apache Airflow totalmente gerenciado em paridade com o Amazon MWAA ou o Cloud Composer do GCP. O equivalente Azure foi estimado com base em Azure Container Apps, que é uma plataforma de contêineres serverless, que reduz artificialmente o custo do Cenário B e não inclui os custos operacionais de configuração, atualização e monitoramento que uma equipe técnica precisaria absorver.

É necessário ressaltar que a comparação não é inteiramente semelhante. Os serviços gerenciados oferecem acordos de nível de serviço (*SLAs*) de disponibilidade, escalabilidade automática, atualizações de segurança aplicadas pela própria provedora e monitoramento integrado. Na solução *open source*, esses mesmos fatores, como atualização de contêineres, reinicialização após falhas, *backup* do PostgreSQL, renovação de certificados TLS, são responsabilidade da equipe interna. O Custo Total de Propriedade (*Total Cost of Ownership*, TCO) depende, portanto, do custo da hora de trabalho da equipe técnica. Para organizações com equipes qualificadas, o *open source* permanece substancialmente mais barato. Para organizações sem colaboradores técnicos dedicados, o valor agregado dos serviços gerenciados pode justificar o custo adicional.

5.2 Análise Crítica e Lições Aprendidas

5.2.1 Limitações de Escalabilidade das Ferramentas *Open Source*

A solução desenvolvida demonstrou viabilidade técnica e econômica para o escopo escolhido. Contudo, a escolha por ferramentas *open source* auto-hospedadas implica limitações de escalabilidade que é necessário delimitar com precisão, para que futuros usuários possam antecipar os pontos de quebra da arquitetura.

O LocalExecutor do Apache Airflow opera exclusivamente no processo do agendador e não distribui tarefas para múltiplos nós de computação. Para os 16 fluxos sequenciais do protótipo essa limitação é irrelevante, mas qualquer ampliação substancial no número de *scrapers*, ou a necessidade de execução paralela de múltiplas DAGs, exigirá migração para o CeleryExecutor ou o KubernetesExecutor, ambos com custos de infraestrutura e configuração muito maiores.

O PostgreSQL está implantado como contêiner único, sem replicação ou mecanismo de *failover* automático. Uma falha no volume de dados ou no próprio contêiner interrompe imedi-

atamente a disponibilidade do *data warehouse*. Em cenários de produção que demandem alta disponibilidade, seria necessário implementar *streaming replication* nativo do PostgreSQL ou soluções de orquestração de *failover*.

O Apache Superset, na configuração padrão com SQLite como banco de metadados, não foi projetado para ambientes multiusuário de alta concorrência. Para *dashboards* acessados por muitos usuários simultâneos ou com consultas SQL complexas sobre grandes volumes de dados, o tempo de resposta pode diminuir progressivamente. A escalabilidade horizontal do Superset requer múltiplos *workers* Celery, um broker Redis dedicado e migração do banco de metadados para PostgreSQL externo, o que aproxima a complexidade operacional da solução *open source* com a de um serviço gerenciado na nuvem.

O Docker Compose foi projetado para orquestração em um único hospedeiro e não suporta distribuição de contêineres entre múltiplos *nodes* de forma nativa. A migração para um ambiente multihost exigiria a inclusão de Kubernetes, introduzindo uma nova camada de complexidade de infraestrutura e, conseqüentemente, um aumento massivo no custo de manutenção.

Em conjunto, essas limitações não invalidam a abordagem para o escopo do projeto, dezenas de fontes, execuções diárias e volumes na casa dos gigabytes, mas definem com clareza o horizonte da solução atual. O crescimento além desses limites demandará ou a evolução das ferramentas *open source* para configurações distribuídas, ainda que dentro do ecossistema de código aberto e com custo substancialmente inferior ao dos serviços gerenciados, como evidenciado na Seção 4.5, ou até mesmo a transição gradual para serviços gerenciados em nuvem, cuja decisão deve ser orientada pelo líder técnico e pela capacidade da equipe responsável pela manutenção.

5.2.2 Síntese das Lições Aprendidas

O desenvolvimento evidenciou que a viabilidade de um *pipeline* de dados *open source* para extração e análise de informações de mercado turístico é técnica e economicamente comprovada. As principais lições aprendidas, com valor imediato para implementações futuras, são organizadas abaixo.

As diferentes formas das fontes é o principal desafio de engenharia. Os 10 *scrapers* desenvolvidos implementam ao menos seis estratégias distintas de extração, *scroll* infinito, carrossel, paginação, botão “ver mais”, *dropdown* de navegação e navegação multi-nível, porque cada operadora turística construiu seu site de forma independente. Essa diversidade torna impossível uma solução genérica a baixo custo, e reforça a necessidade de manutenção contínua, alterações

no *layout* de qualquer site exigem atualização do *scraper* correspondente.

A deduplicação incremental é uma funcionalidade, não um detalhe. A estratégia de comparar tuplas de campos-chave antes de inserir registros, implementada via `get_existing_records()` no `db_manager.py`, permite que os *scrapers* sejam reexecutados sem gerar duplicatas, o que é essencial tanto para a confiabilidade dos dados quanto para a operação de recuperação após falhas.

O LocalExecutor do Airflow é suficiente para o escopo atual, mas cria débito arquitetural. A escolha deliberada pelo LocalExecutor simplificou enormemente a infraestrutura inicial, mas qualquer incremento substancial no número de *scrapers* exigirá migração para um executor distribuído. Documentar essa decisão e seus *trade-offs* desde o início é fundamental para que futuros mantenedores compreendam o contexto da escolha, estejam cientes do débito técnico.

Ferramentas *open source* não eliminam custos, mas os redistribuem. O custo de licenciamento zero é substituído por custos de operação (servidor), custo de aprendizado (curva de adoção das ferramentas) e custo de manutenção (atualizações, correções de *scrapers*, monitoramento). Para organizações sem equipe técnica dedicada, a comparação com soluções SaaS deve considerar o custo total de propriedade (*Total Cost of Ownership* ou TCO), e não apenas o custo de licença.

A separação dos dados em camadas demonstrou seu valor prático. O modelo de quatro camadas implementado, *scrapers*, passando pelo DuckDB/PostgreSQL *raw*, depois pelos modelos dbt staging/marts, até o Superset, permitiu que cada componente evoluísse de forma independente. A adição de novos *scrapers* não afetou os modelos dbt existentes, e a refatoração dos modelos não exigiu alterações nos *scrapers*. Essa independência é um resultado direto da adesão ao princípio de separação de responsabilidades e ao padrão de dados em camadas (*medallion architecture* simplificada).

Em resumo, o *pipeline* desenvolvido demonstra que é possível construir uma infraestrutura analítica funcional, auditável e economicamente acessível inteiramente com ferramentas de código aberto. As limitações identificadas não invalidam a abordagem, mas delimitam com precisão seu envelope de aplicabilidade. O protótipo é adequado para dezenas de fontes e volumes na casa dos gigabytes, e requer evolução arquitetural planejada para os próximos patamares de escala.

Referências Bibliográficas

- Amazon Web Services. AWS Pricing. <https://aws.amazon.com/pricing/>, 2026. Acessado em maio de 2026.
- Chaimae Boulahia, Hicham Behja, Mohammed Reda Chbihi Louhdi, and Zoubair Boulahia. The multi-criteria evaluation of research efforts based on etl software. *Evolutionary Intelligence*, 17:2099–2124, 2024. doi: 10.1007/s12065-023-00899-z.
- Bibhu Dash and Swati Swayamsiddha. Reverse etl for improved scalability, observability, and performance of modern operational analytics - a comparative review. In *2022 OITS International Conference on Information Technology (OCIT)*. IEEE, 2022. doi: 10.1109/OCIT56763.2022.00097.
- Maureen Dobbins. Rapid review guidebook. *Natl Collab Cent Method Tools*, 13:25, 2017.
- Google Cloud. Google Cloud Pricing. <https://cloud.google.com/pricing>, 2026. Acessado em maio de 2026.
- Philip Heltweg and Dirk Riehle. A systematic analysis of problems in open collaborative data engineering. *ACM Transactions on Social Computing*, 6(3-4):1–30, 2023.
- Moaiaad Ahmad Khder. Web scraping or web crawling: State of art, techniques, approaches and application. *International Journal of Advances in Soft Computing & Its Applications*, 13(3), 2021.
- Anthony Mbata, Yaji Sripada, and Mingjun Zhong. A survey of pipeline tools for data engineering. *arXiv preprint arXiv:2406.08335*, 2024.
- Microsoft Azure. Azure Pricing. <https://azure.microsoft.com/en-us/pricing>, 2026. Acessado em maio de 2026.
- Sachin Murarka, Anshuj Jain, and Laxmi Singh. Advanced techniques in data ingestion and pipelining for scalable big data platforms. In *ICTBIG 2024*. IEEE, 2024.
- Joshua Chibuike Nwokeji and Richard Matovu. A systematic literature review on big data extraction, transformation and loading (etl). In *Lecture Notes in Computer Science*. Springer, 2021. doi: 10.1007/978-3-030-80126-7_24.

- Ken Peffers, Marcus Rothenberger, Tuure Tuunanen, and Reza Vaezi. Design science research evaluation. In *International Conference on Design Science Research in Information Systems*, pages 398–410. Springer Berlin Heidelberg, 2012. doi: 10.1007/978-3-642-29863-9_33.
- Carolline Pena, Bruno Cartaxo, Igor Steinmacher, Deepika Badampudi, Deyvson da Silva, Williby Ferreira, Adauto Almeida, Fernando Kamei, and Sergio Soares. Comparing the efficacy of rapid review with a systematic review in the software engineering field. *Journal of Software: Evolution and Process*, 37(1):e2748, 2025.
- Vijay Janapa Reddi, Greg Damos, Pete Warden, Peter Mattson, and David Kanter. Data engineering for everyone. *arXiv preprint arXiv:2102.11447*, 2021.
- Ridwan Fadjar Septian, Fajri Abdillah, and Tajhul Faijin Aliyudin. A study review of common big data architecture for small-medium enterprise. In *MSCEIS 2019*, 2019. doi: 10.4108/eai.12-10-2019.2296535.